INFRASTRUCTURE & DEVOPS

Observability & Production Debugging

Logs, metrics, traces, profiles, SLOs and incident response — instrumenting systems you will not be around to watch.

observability · monitoring · prometheus · promql · grafana · opentelemetry · distributed-tracing · structured-logging · slo · error-budget · alerting · incident-response · postmortem · profiling · llm-observability · fde

Study Pack · Comprehensive reference & interview prep · ~133 min read · 29,308 words

CONTENTS

1. Monitoring Versus Observability
2. The Three Pillars — and the Fourth
3. Structured Logging Done Right
4. Metrics Fundamentals
5. Prometheus in Practice
6. Grafana Dashboards People Actually Use
7. Distributed Tracing
8. OpenTelemetry End to End
9. SLIs, SLOs and Error Budgets
10. Alerting That Does Not Cry Wolf
11. Debugging Distributed Systems
12. Profiling and Performance
13. LLM and AI Observability
14. Incident Response and Incident Command
15. Blameless Postmortems
16. Observability Inside a Customer's Environment
17. Interview Questions
18. Instrumentation Checklist and Lab Plan

Monitoring Versus Observability

Monitoring answers questions you already knew to ask. Observability answers questions you did not know to ask until three in the morning, when a customer's integration is failing for 4% of requests and you are on a video call with someone whose job title you cannot pronounce. As a Forward Deployed Engineer the distinction is not academic: you install software into an environment you do not own, you cannot SSH into it next month, and you will not be sitting in front of their dashboard when it breaks. Everything you fail to instrument before you leave becomes a phone call, a screen share, and a credibility problem.

Known
UNKNOWNS = MONITORING

Unknown
UNKNOWNS = OBSERVABILITY

<5%
CPU OVERHEAD BUDGET FOR TELEMETRY

6 wks
TYPICAL TIME YOU HAVE ON SITE

A definition you can actually use

The control-theory definition — "a system is observable if its internal state can be inferred from its outputs" — is correct but useless at 3am. The operational definition is better: **a system is observable if you can answer a new question about its behaviour without shipping new code.** That is the whole test. If answering "which tenant is generating the slow requests?" requires you to add a log line, rebuild, get a change-board approval and wait nine days for the customer's next release window, your system is not observable. It is monitored.

Dimension	Monitoring	Observability
Question shape	"Is X above threshold?"	"Why is X behaving like that?"
Defined	In advance, by you	At query time, by whoever is debugging
Data shape	Pre-aggregated, low cardinality	High-cardinality, high-dimensional events
Typical artefact	Dashboard, alert rule	Trace, ad-hoc query, exemplar
Fails when	The failure is novel	You did not propagate context
Cost driver	Number of series	Volume and retention of events

> **Both, not either**
>
> This is not a replacement argument. You need monitoring for the twenty failure modes you can predict (disk full, certificate expiring, queue backing up, pod crash-looping) because those produce cheap, reliable, actionable alerts. You need observability for the eighty you cannot. A mature deployment has a small number of high-quality alerts backed by a large, queryable body of telemetry.

Why this is harder for an FDE than for a SaaS team

A SaaS team owns one production environment. They choose the vendor, they have admin on the dashboard, they can deploy a fix in twenty minutes, and they see every incident. You have none of that. Internalise these constraints now, because every technical decision in this guide bends around them.

YOU DO NOT OWN THE BACKEND

The customer already has Splunk, or Datadog, or Elastic, or Dynatrace, or a homegrown Graylog cluster. You will be asked to ship into it. Your instrumentation must be vendor-neutral or you will be rewriting it per customer.

YOU CANNOT SEE THEIR DATA

Many customers will not grant you read access to production telemetry. You will debug through someone else's screen share, or through exported CSVs. Design your output to be legible to a stranger reading it aloud.

EGRESS MAY BE FORBIDDEN

Air-gapped, government, healthcare and defence environments block outbound traffic entirely. Any telemetry design that assumes "send to our cloud" is dead on arrival. Plan for local-only collection with manual export.

CHANGES ARE SLOW

Adding a log line can mean a release, a CAB ticket and a maintenance window. Instrument richly up front; the marginal cost of an extra attribute at build time is near zero, and the marginal cost of adding one later is a fortnight.

YOU WILL LEAVE

In six weeks the people operating this are the customer's L1 team. If a dashboard needs you to explain it, it is broken. If an alert has no runbook, it will be muted within a month.

DATA IS SENSITIVE

Their logs contain their customers' PII, their contract values, their patient identifiers. One careless `log.info(request.body)` and you have created a regulatory incident on their estate with your name on the commit.

The question-driven instrumentation method

Before you write a single instrumentation call, write down the questions. This takes forty minutes and is the highest-leverage activity in the whole discipline. For a document-ingestion pipeline deployed at an insurer, the list looks like this:

1. **Is it working at all?**
   Requests per second, success rate, and time since last successful end-to-end run. If you only get one signal, get this one.

2. **Is it working for everyone?**
   Break down by tenant, by region, by document type, by upstream system. Aggregate health hides single-customer outages — and the single customer who is broken is the one who escalates.

3. **Is it fast enough?**
   p50, p95, p99 latency by operation. Averages are a lie; the mean of 100 fast requests and one 30-second request looks fine.

4. **Where is the time going?**
   Per-stage duration: fetch, OCR, embed, retrieve, generate, write. Without this you cannot tell "the LLM is slow" from "their SharePoint is slow".

5. **What changed?**
   Deploy markers, config changes, feature flag flips, certificate rotations, upstream version bumps. Most incidents are caused by a change; most of the first thirty minutes is spent discovering which one.

6. **What will break next?**

Queue depth, disk free, token budget consumed, certificate expiry, licence expiry, connection-pool saturation.

7 **Who do I call?**

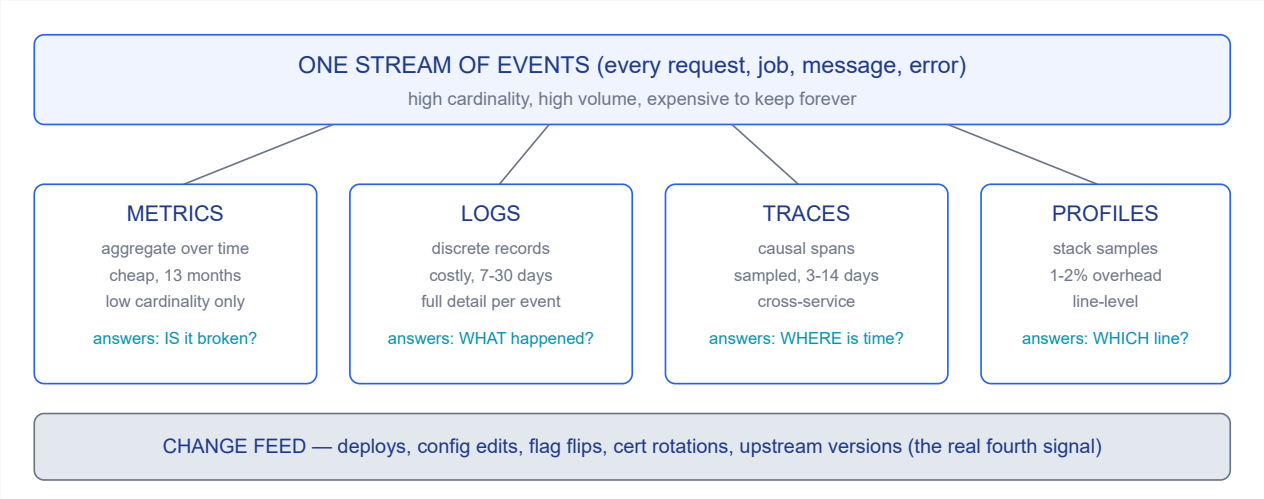
Every alert maps to an owner and a runbook. If you cannot name the owner, the alert should not exist.

✦ **Write the dashboard before the code**

Sketch the single screen you want on the wall during go-live. Each panel implies a metric; each metric implies an instrumentation call. Working backwards from the dashboard gets you a coherent telemetry model in an hour. Working forwards from "let me add some metrics" gets you 400 series, none of which answer a question anyone is asking.

The Three Pillars — and the Fourth

Logs, metrics and traces are the canonical three pillars. The framing is useful for learning and slightly misleading in practice: they are not three separate systems so much as three projections of the same underlying stream of events, at different resolutions and different costs. The fourth signal — continuous profiles — closes the loop from "which service is slow" down to "which line of code". And the fifth, which nobody counts but which resolves more incidents than the other four combined, is the change feed.



The pillars are projections of one event stream. Correlation IDs are what let you move between them.

Cost, cardinality and retention — the triangle you cannot escape

Every observability decision is a trade between detail, cost and retention. Metrics are cheap per data point and can be kept for a year, but they cannot carry a user ID. Logs carry everything but cost roughly 50–200× more per byte stored and are usually kept for days. Traces carry causality but are sampled, so the one request you care about is often the one that was thrown away. Profiles are cheap to collect and almost free to store because they are aggregated server-side, but they do not tell you which request caused the CPU burn.

Signal	Typical volume per 1k rps	Typical retention	Relative storage cost	Cardinality ceiling
Metrics	~2–5k active series	13 months	1×	Low — hundreds of label values
Logs	50–500 GB/day	7–30 days hot	50–200×	Unbounded
Traces (1% sampled)	5–20 GB/day	3–14 days	20–60×	Unbounded
Profiles	<1 GB/day	14–30 days	2–5×	By service/version only
Change events	Tens per day	Forever	~0	N/A

> ⚠ **The number that gets you fired**
> A customer's Datadog or Splunk bill is a line item somebody defends to a CFO. If your service starts emitting a metric labelled with `user_id` and they have 2 million users, you have created 2 million time series. At typical custom-metric pricing that is a five-figure monthly surprise, and the remediation is to delete your integration. Always ask: "what is the maximum number of distinct values this label can take, forever?" If the answer is unbounded, it is an attribute on a log or a span, never a metric label.

Wide events — the pragmatic modern synthesis

The highest-signal pattern available today is deceptively simple: emit **one wide, structured event per unit of work**, with every piece of context you have, and derive the rest. One request produces one JSON object with forty or eighty fields: identifiers, timings for each stage, outcome, versions, feature flags, tenant, upstream latencies, retry counts, cache hit ratios. From that single stream you can compute metrics, you can search like logs, and with span context you can reconstruct traces.

JSON

```
{
  "ts": "2026-03-14T09:21:07.411Z",
  "event": "http_request",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "service": "doc-ingest",
  "version": "2.7.1",
  "build_sha": "a91f3c2",
  "env": "prod",
  "region": "eu-central-1",
  "pod": "doc-ingest-7f9c8-4xq2l",
  "tenant_id": "acme-insurance",
  "user_hash": "sha256:9f2b … e41",
  "route": "POST /v1/documents",
  "status": 200,
  "duration_ms": 1843,
  "db_ms": 42,
  "sharepoint_ms": 1210,
  "ocr_ms": 455,
  "embed_ms": 96,
  "queue_wait_ms": 12,
  "retries": 1,
  "retry_reason": "sharepoint_429",
  "cache": "miss",
  "doc_pages": 14,
  "doc_bytes": 2841233,
  "doc_mime": "application/pdf",
  "flags": ["ocr_v2", "chunk_semantic"],
  "upstream_version": "SP-2019-CU22"
}
```


Notice what is *not* in there: no document contents, no user email, no raw file name (which in insurance routinely contains a claimant's surname). Notice what is: `sharepoint_ms` separate from `duration_ms`, which is what lets you say "we are not slow, your SharePoint is taking 1.2 seconds per fetch" with evidence rather than opinion — a sentence that saves entire projects.

✓ **The FDE superpower**

Per-dependency timing on every event. The single most common conversation in an enterprise deployment is "your thing is slow". The only way to win it is a chart that shows your processing time flat at 90ms while the customer's identity provider, file share or ERP fluctuates between 400ms and 8 seconds. Instrument every outbound call with its own duration, its own status, and the upstream system's name.

Structured Logging Done Right

Unstructured logs are a confession that you expect a human to read every line. Nobody reads every line. At any realistic volume, logs are a database that you query, and a database with no schema is a swamp. Structured logging — one JSON object per event, consistent field names, machine-parseable — costs you nothing at write time and transforms what is possible at read time.

The case, in one comparison

TEXT

```
UNSTRUCTURED — what most codebases emit
2026-03-14 09:21:07 INFO  Processing document for user john.doe@acme.com (claim CLM-88
2026-03-14 09:21:09 ERROR Failed: timeout

  To find "how many documents over 10 pages failed for tenant acme last week"
  you must write a regex, and the regex breaks the day someone edits the message.
  The email address is now permanently in the customer's log store. The error
  line has no identifiers, so you cannot join it to the request that caused it.

STRUCTURED — same two events
{"ts":"2026-03-14T09:21:07.411Z","level":"info","event":"doc.process.start",
 "trace_id":"4bf92f35 … ","tenant_id":"acme","doc_id":"d-91f2","pages":14}
{"ts":"2026-03-14T09:21:09.882Z","level":"error","event":"doc.process.failed",
 "trace_id":"4bf92f35 … ","tenant_id":"acme","doc_id":"d-91f2","pages":14,
 "error_type":"UpstreamTimeout","upstream":"sharepoint","timeout_ms":2000,
 "duration_ms":2471,"attempt":3}

  The query is now: event="doc.process.failed" AND tenant_id="acme" AND pages>10
  No regex. No PII. The trace_id joins it to the full request.
```

Levels, and what each one actually means

Log levels are a contract with whoever is on call. The most common failure is level inflation: everything becomes `ERROR`, so `ERROR` means nothing, so people stop looking. Hold the line on these definitions.

Level	Means	Should it page?	Example
`TRACE`	Firehose for local debugging	Never ship it on	Every loop iteration, raw payload shapes
`DEBUG`	Developer detail, off in prod by default	No	"cache miss for key X, fetching"
`INFO`	A notable thing happened as designed	No	"service started", "job completed", "config loaded"
`WARN`	Degraded but handled; would be a problem if sustained	Only in aggregate	"retry 2 of 3", "fell back to cached result", "cert expires in 14d"
`ERROR`	This unit of work failed; a human may need to act	Rate-based, yes	"could not write to database after 3 attempts"
`FATAL` / `CRITICAL`	The process cannot continue and is exiting	Yes, immediately	"required config MISSING, refusing to start"

⚠ **The handled-exception trap**

`ERROR` for something you successfully retried is the single biggest source of alert fatigue. If attempt 1 failed and attempt 2 succeeded, the unit of work *succeeded*: log the first attempt at `WARN` with `attempt=1`, and log the success at `INFO` with `retries=1`. Reserve `ERROR` for terminal failure of the whole operation. Then "error rate" means something and you can alert on it.

What to log — and what must never appear

Assume every log line you write will be retained for 30 days in a system that 400 people at the customer can read, that is replicated to a third-party vendor, and that will be exported in response to a legal discovery request. That is not hyperbole; it is literally how enterprise log platforms work.

ALWAYS LOG

Correlation IDs (trace, request, job). Tenant and environment. Service name and version/build SHA. The operation name as a stable enum. Duration. Outcome. Error type and a stable error code. Upstream system name and its latency. Retry count. Resource identifiers that are opaque (`doc_id`, not `file_name`).

NEVER LOG

Passwords, tokens, API keys, session cookies, Authorization headers. Full request/response bodies. Email addresses, phone numbers, national IDs, card numbers, patient identifiers. Customer file names and document contents. Database connection strings. LLM prompts containing user data. Anything in a field you did not personally audit.

PYTHON

```python
# ─────────────────────────────────────────────────────────────────
# structlog: production configuration for a service you will hand over
# ─────────────────────────────────────────────────────────────────
import logging, os, re, sys, uuid, contextvars
import structlog

request_id_var: contextvars.ContextVar[str] = contextvars.ContextVar("request_id", def
tenant_var: contextvars.ContextVar[str] = contextvars.ContextVar("tenant", default="-"

SECRET_KEYS = {"password", "passwd", "secret", "token", "api_key", "apikey",
               "authorization", "cookie", "set-cookie", "client_secret",
               "private_key", "refresh_token", "access_token", "ssn", "pan"}

EMAIL_RE = re.compile(r"[\w.+-]+@[\w-]+\.[\w·-]+")
CARD_RE  = re.compile(r"\b(?:\d[ -]*?){13,19}\b")

def redact(_logger, _method, event_dict):
    """Defence in depth: strip secrets even if a caller is careless."""
    for k in list(event_dict):
        if k.lower() in SECRET_KEYS:
            event_dict[k] = "[REDACTED]"
        elif isinstance(event_dict[k], str):
            v = EMAIL_RE.sub("[EMAIL]", event_dict[k])
            v = CARD_RE.sub("[CARD]", v)
            event_dict[k] = v
    return event_dict

def add_context(_logger, _method, event_dict):
    event_dict["request_id"] = request_id_var.get()
    event_dict["tenant_id"]  = tenant_var.get()
    event_dict["service"]    = os.getenv("OTEL_SERVICE_NAME", "doc-ingest")
    event_dict["version"]    = os.getenv("APP_VERSION", "dev")
    event_dict["env"]        = os.getenv("APP_ENV", "local")
    # Join logs to traces: without this, your trace view and log view are two islands
    try:
        from opentelemetry import trace
        ctx = trace.get_current_span().get_span_context()
        if ctx.is_valid:
            event_dict["trace_id"] = format(ctx.trace_id, "032x")
            event_dict["span_id"]  = format(ctx.span_id, "016x")
    except Exception:
        pass
    return event_dict

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.stdlib.add_log_level,
```

```
        structlog.processors.TimeStamper(fmt="iso", utc=True, key="ts"),
        add_context,
        redact,
        structlog.processors.StackInfoRenderer(),
        structlog.processors.format_exc_info,
        # Humans get colour locally; machines get JSON everywhere else
        structlog.dev.ConsoleRenderer() if sys.stderr.isatty()
            else structlog.processors.JSONRenderer(sort_keys=False),
    ],
    wrapper_class=structlog.make_filtering_bound_logger(
        getattr(logging, os.getenv("LOG_LEVEL", "INFO").upper())),
    cache_logger_on_first_use=True,
)

log = structlog.get_logger()
```

PYTHON

```python
# ──────────────────────────────────────────────────────────────────────
# Using it: one wide event per unit of work, not a trail of breadcrumbs
# ──────────────────────────────────────────────────────────────────────
import time
from contextlib import contextmanager

@contextmanager
def timed(bucket: dict, key: str):
    t0 = time.perf_counter()
    try:
        yield
    finally:
        bucket[key] = round((time.perf_counter() - t0) * 1000, 1)

def process_document(doc_id: str, tenant: str) -> dict:
    request_id_var.set(str(uuid.uuid4()))
    tenant_var.set(tenant)
    timings: dict[str, float] = {}
    t0 = time.perf_counter()
    attempt = 0

    try:
        with timed(timings, "fetch_ms"):
            blob, meta = sharepoint.fetch(doc_id)          # may raise
        with timed(timings, "ocr_ms"):
            text = ocr.extract(blob)
        with timed(timings, "embed_ms"):
            vectors = embedder.embed(chunk(text))
        with timed(timings, "index_ms"):
            index.upsert(doc_id, vectors)

        log.info("doc.processed",
                 event_name="doc.processed",
                 doc_id=doc_id,
                 pages=meta.pages,
                 bytes=len(blob),
                 mime=meta.mime,
                 chunks=len(vectors),
                 upstream="sharepoint",
                 upstream_version=meta.server_version,
                 retries=attempt,
                 outcome="success",
                 duration_ms=round((time.perf_counter() - t0) * 1000, 1),
                 **timings)
        return {"ok": True}

    except UpstreamTimeout as e:
        log.error("doc.failed",
```

```
                doc_id=doc_id,
                error_type="UpstreamTimeout",
                error_code="E_SP_TIMEOUT",
                upstream="sharepoint",
                timeout_ms=e.timeout_ms,
                retries=attempt,
                outcome="failed",
                duration_ms=round((time.perf_counter() - t0) * 1000, 1),
                **timings)
        raise
    except Exception:
        # exc_info gives you the stack; the formatter above renders it as a field
        log.exception("doc.failed", doc_id=doc_id, error_code="E_UNEXPECTED",
                      outcome="failed", **timings)
        raise
```

✦ **Stable error codes earn their keep**

Give every failure mode a short stable code — `E_SP_TIMEOUT`, `E_SP_403`, `E_OCR_CORRUPT`, `E_LLM_RATE_LIMIT`. The customer's L1 team can then be handed a one-page table mapping code to first action, and a search for a code returns every occurrence regardless of how the message text has drifted across versions. It also makes alerting precise: you can page on `E_SP_403` (credential problem, needs a human now) while merely counting `E_SP_TIMEOUT` (transient, retried).

The Node.js equivalent

JAVASCRIPT

```ts
// pino: fastest structured logger in Node, ~5x the throughput of winston
import pino from 'pino';
import { AsyncLocalStorage } from 'node:async_hooks';
import { randomUUID } from 'node:crypto';
import { trace } from '@opentelemetry/api';

export const als = new AsyncLocalStorage();

const REDACT_PATHS = [
  'req.headers.authorization', 'req.headers.cookie', 'res.headers["set-cookie"]',
  '*.password', '*.token', '*.apiKey', '*.api_key', '*.client_secret',
  '*.refresh_token', '*.access_token', 'body', 'payload',
];

export const log = pino({
  level: process.env.LOG_LEVEL ?? 'info',
  base: {
    service: process.env.OTEL_SERVICE_NAME ?? 'doc-api',
    version: process.env.APP_VERSION ?? 'dev',
    env:     process.env.APP_ENV ?? 'local',
  },
  timestamp: pino.stdTimeFunctions.isoTime,
  // pino's redact is applied at serialisation time; it is fast and total
  redact: { paths: REDACT_PATHS, censor: '[REDACTED]' },
  formatters: {
    level: (label) => ({ level: label }),          // "level":"info" not "level":30
    log: (obj) => {
      const store = als.getStore();
      const span  = trace.getActiveSpan()?.spanContext();
      return {
        ...obj,
        request_id: store?.requestId,
        tenant_id:  store?.tenantId,
        ...(span?.traceId ? { trace_id: span.traceId, span_id: span.spanId } : {}),
      };
    },
  },
});

// Express middleware: one wide event per request, emitted on finish
export function requestLogger(req, res, next) {
  const requestId = req.headers['x-request-id'] ?? randomUUID();
  const tenantId  = req.headers['x-tenant-id'] ?? 'unknown';
  res.setHeader('x-request-id', requestId);          // echo it back - vital for supp
  const start = process.hrtime.bigint();

  als.run({ requestId, tenantId }, () => {
    res.on('finish', () => {
```

```
      const durationMs = Number(process.hrtime.bigint() - start) / 1e6;
      const level = res.statusCode ≥ 500 ? 'error'
                  : res.statusCode ≥ 400 ? 'warn' : 'info';
      log[level]({
        event: 'http_request',
        method: req.method,
        route: req.route?.path ?? req.path,   // route, NOT raw path: /users/:id
        status: res.statusCode,
        duration_ms: Math.round(durationMs * 10) / 10,
        bytes_out: Number(res.getHeader('content-length') ?? 0),
        ua_family: shortUserAgent(req.headers['user-agent']),
        upstream_ms: res.locals.upstreamMs ?? 0,
      }, 'request completed');
    });
    next();
  });
}
```

✕ **Route, never raw path**

`/users/48213/orders/99f2` as a log field or metric label is a cardinality bomb and a privacy risk. Always log the *route template* — `/users/:id/orders/:orderId` — and put the identifiers in separate explicit fields where you control them. The same rule applies to span names and to Prometheus labels. Frameworks give you this for free (`req.route.path` in Express, `request.url.path_format` or the matched route in FastAPI/Starlette); use it.

Correlation IDs: the thread through everything

A correlation ID is a value generated once at the edge and propagated through every log line, every downstream HTTP call, every queue message and every database comment, so that a single search returns the whole story of one request across six services. If you adopt one practice from this section, adopt this one — it is the difference between a ten-minute diagnosis and a two-day one.

1. **Accept an inbound ID if present.**
   Honour `traceparent` (W3C), and as a fallback `X-Request-ID` or `X-Correlation-ID`. Enterprise API gateways and load balancers often already inject one; reusing it means their gateway logs join to yours, which makes you look like you planned it.

2. **Generate one if absent.**
   A UUIDv4, or better the 32-hex trace ID so it is the same identifier as your tracing system uses.

3. **Store it in ambient context.**
   `contextvars` in Python, `AsyncLocalStorage` in Node, MDC in Java/Logback, `context.Context` in Go. Never pass it manually through forty function signatures; you will

miss one.

4. **Attach it to every log line automatically.**
Via the formatter, as shown above — not at each call site.

5. **Propagate it outbound.**
On every HTTP client call, every Kafka/RabbitMQ message header, every job enqueue. Use an instrumented HTTP client so this is not optional.

6. **Return it to the caller.**
Echo it in a response header and show it in any user-facing error page: "Something went wrong. Reference: 4bf92f35." A support ticket that contains the trace ID is resolved in minutes; one that says "it was broken this morning" takes a day.

PYTHON

```python
# FastAPI: accept-or-generate, propagate, echo, and put it in DB comments
from fastapi import FastAPI, Request
from opentelemetry import trace
import httpx

app = FastAPI()
# One shared client with connection pooling AND trace propagation baked in
client = httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=2.0),
                           limits=httpx.Limits(max_connections=100,
                                               max_keepalive_connections=20))

@app.middleware("http")
async def correlate(request: Request, call_next):
    incoming = (request.headers.get("traceparent", "").split("-")[1:2] or [None])[0]
    rid = incoming or request.headers.get("x-request-id") or uuid.uuid4().hex
    token = request_id_var.set(rid)
    try:
        response = await call_next(request)
    finally:
        request_id_var.reset(token)
    response.headers["x-request-id"] = rid
    return response

async def call_crm(path: str):
    # httpx + opentelemetry-instrumentation-httpx injects traceparent automatically;
    # we add the human-readable header too because many enterprise gateways log it.
    return await client.get(path, headers={"x-request-id": request_id_var.get()})

def run_query(conn, sql, params):
    # SQL comment propagation (sqlcommenter style). When the customer's DBA sends you
    # the slow query log, each statement carries the request that issued it.
    rid = request_id_var.get()
    return conn.execute(f"{sql} /* request_id='{rid}',service='doc-ingest' */", params
```

Sampling logs without losing the plot

At 5,000 requests per second, one INFO line per request is 432 million lines a day. Someone will ask you to turn it off. Sample instead, and sample *intelligently*:

KEEP ALL ERRORS

Never sample `ERROR` or `WARN`. They are rare by definition and they are the ones you need.

KEEP ALL SLOW REQUESTS

Anything over your p99 target is interesting by construction. `if duration_ms > 1000: always_log()`.

SAMPLE THE BORING MAJORITY

Fast 200s: keep 1 in 100, and record the sample rate in the event (`"sample_rate":100`) so counts can be reconstructed by multiplying.

HEAD-BASED, PER TRACE

Decide once per trace and propagate the decision, so you never keep the frontend span and drop the backend span of the same request.

GUARANTEE PER-TENANT COVERAGE

Keep at least N events per tenant per minute, so a small customer with 3 rps is not statistically erased by a large one with 3,000.

DYNAMIC DEBUG LEVEL

Expose an admin endpoint or SIGHUP handler to raise log level for 15 minutes without a restart. Then auto-revert — a forgotten DEBUG level has filled more disks than any bug.

PYTHON

```python
import random

def should_log(status: int, duration_ms: float, level: str) -> tuple[bool, int]:
    """Return (keep, sample_rate). sample_rate is recorded so counts stay correct."""
    if level in ("error", "warning") or status ≥ 500:
        return True, 1
    if duration_ms > SLOW_MS:                  # everything above the SLO threshold
        return True, 1
    if status ≥ 400:
        return random.random() < 0.10, 10      # client errors: 10%
    return random.random() < 0.01, 100         # happy path: 1%

# Reconstructing true counts later:
#   SELECT SUM(sample_rate) FROM events WHERE status = 200   -- not COUNT(*)
```

> **Log to stdout, always**
> In a container, write JSON to stdout and stop there. Do not write files, do not rotate logs, do not ship from inside the process, do not open a TCP socket to a log server. The platform — Docker, Kubernetes, systemd, the customer's agent — collects stdout. A process that writes its own log files in a container will eventually fill an ephemeral volume and take the node down with it, and the file will be invisible to the customer's collector anyway. The one exception: air-gapped deployments where you must write a rotating file for manual collection, in which case use the platform's logrotate with a hard size cap and document it.

Metrics Fundamentals

Metrics are numbers measured over time, aggregated at write time so they are cheap to store and fast to query. They are what dashboards and alerts are built from. Their power is that you can keep thirteen months of them for the price of a day of logs; their limitation is that aggregation is lossy and irreversible — once you have added a request into a counter you can never get that individual request back. Understanding the four metric types, and the cardinality rules that govern them, is the difference between a telemetry design that scales and one that gets your agent uninstalled.

The four types

Type	Behaviour	You query it with	Use for	Never use for
Counter	Monotonically increasing; resets to 0 on restart	`rate()`, `increase()`	Requests, errors, bytes, messages, retries, tokens	Anything that can go down
Gauge	Goes up and down; a snapshot of "now"	Raw value, `avg`, `max`, `delta()`	Queue depth, in-flight requests, memory, temperature, pool connections	Rates (comput those from counters)
Histogram	Counts observations into predefined buckets, plus sum and count	`histogram_quantile()`	Latency, payload size, batch size, token counts	High-cardinalit labels — bucke multiply
Summary	Client-side computed quantiles, plus sum and count	Direct quantile series	Single-instance quantiles where you cannot pick buckets	Anything you need to aggregate across instance

⚠ **Why summaries lose to histograms**

You cannot average percentiles. If instance A reports p99 = 100ms and instance B reports p99 = 900ms, the fleet p99 is *not* 500ms — it is unknowable from those two numbers. Summaries compute quantiles on the client, so they cannot be re-aggregated. Histograms ship raw bucket counts, which *can* be summed across instances and then inverted into a quantile. In any system with more than one replica — which is all of them — use histograms. The cost is that you must choose buckets in advance.

Choosing histogram buckets

Default buckets in most libraries are tuned for sub-second HTTP APIs: `.005, .01, .025, .05, .1, .25, .5, 1, 2.5, 5, 10`. They are wrong for almost every enterprise workload you will instrument. If your document pipeline takes 20–90 seconds, every observation lands in the `+Inf` bucket and

`histogram_quantile` returns garbage. If your cache lookup takes 300 microseconds, everything lands in the first bucket and p50 = p99 = 5ms.

PYTHON

```python
from prometheus_client import Histogram, Counter, Gauge

# Rule: buckets should straddle your SLO, with 4-6 below it and 3-4 above.
# SLO = 2s for document ingest, with a long tail up to ~2 minutes.
DOC_LATENCY = Histogram(
    "doc_process_duration_seconds",
    "End-to-end document processing time",
    labelnames=("stage", "doc_type", "outcome"),
    buckets=(0.1, 0.25, 0.5, 1, 2, 3, 5, 10, 30, 60, 120, float("inf")),
)

# Fast internal operation: microsecond-scale buckets
CACHE_LOOKUP = Histogram(
    "cache_lookup_duration_seconds", "Cache lookup time",
    buckets=(0.0001, 0.00025, 0.0005, 0.001, 0.0025, 0.005, 0.01, 0.05),
)

# Counters: name ends in _total, unit in the name, errors get a reason label
DOC_TOTAL = Counter("doc_process_total", "Documents processed",
                    ["doc_type", "outcome", "tenant_tier"])
UPSTREAM_ERRORS = Counter("upstream_errors_total", "Failed upstream calls",
                          ["upstream", "error_code"])

# Gauges: present state only
QUEUE_DEPTH  = Gauge("ingest_queue_depth", "Messages awaiting processing", ["queue"])
IN_FLIGHT    = Gauge("http_requests_in_flight", "Concurrent requests being served")
CERT_EXPIRY  = Gauge("tls_cert_expiry_timestamp_seconds", "Cert notAfter", ["cn"])
```

Bucket arithmetic you should know

A histogram with *B* buckets and *L* label combinations creates `L × (B + 2)` time series (buckets plus `_sum` and `_count`). Twelve buckets, three labels with 5 × 4 × 3 values = 60 combinations → 840 series from one metric. That is fine. The same metric with a `customer_id` label across 500 customers is 420,000 series, which will slow their Prometheus to a crawl and may exceed a vendor's custom-metric quota. Do the multiplication before you ship.

Cardinality: the one way to break someone else's monitoring

Cardinality is the number of distinct time series. Each unique combination of metric name and label values is one series, and each series costs memory in the scraper, disk in the TSDB and money with a vendor. Prometheus holds roughly 3–4 KB of RAM per active series; a million series is 3–4 GB of resident memory before you have queried anything.

TEXT

```
CARDINALITY EXPLOSION — a real incident, reconstructed

  http_request_duration_seconds{
      method="POST",               # 5 values
      route="/v1/documents",       # 40 values
      status="200",                # 6 values
      pod="api-7f9c8-4xq2l",       # 24 values, CHANGES EVERY DEPLOY
      user_id="u_8812394",         # 180,000 values   ⟵ the bomb
  }

  5 x 40 x 6 x 24 x 180000 = 5.18 billion potential series
  Observed: 2.1M active series in 40 minutes. Prometheus OOMKilled at 14 GB.
  Alerting stopped. Nobody noticed for 6 hours because the alert that would
  have told them was evaluated by the Prometheus that died.

  THE FIX
  - user_id   → removed from metrics; kept as a span attribute and log field
  - pod       → dropped via metric_relabel_configs (it is in the target labels
                already, and it churns on every rollout)
  - tenant_id → kept, but bucketed: tenant_tier="enterprise|standard|trial"
  Result: 5 x 40 x 6 x 3 = 3,600 series. Same dashboards. 0.07% of the cost.
```

Labels that are always wrong

User ID, email, session ID, request ID, trace ID, full URL path with identifiers, timestamp, UUID of any kind, error message string, SQL query text, file name, IP address, container ID, pod name (when it churns), git SHA on a high-frequency metric, and anything derived from user input. If you want to slice by one of these, you want a trace or a log, not a metric. The exemplars feature (below) exists precisely to bridge this gap.

YAML

```yaml
# Defensive relabelling — drop high-cardinality labels at scrape time.
# Put this in the customer's Prometheus if you cannot change the application.
scrape_configs:
  - job_name: doc-ingest
    kubernetes_sd_configs: [{ role: pod }]
    metric_relabel_configs:
      # 1. Drop a known-bad label entirely
      - regex: "user_id|session_id|request_id"
        action: labeldrop

      # 2. Drop an entire metric that someone added carelessly
      - source_labels: [__name__]
        regex: "app_debug_.*"
        action: drop

      # 3. Collapse a path label into a route template
      - source_labels: [path]
        regex: "/v1/documents/[0-9a-f-]+"
        target_label: route
        replacement: "/v1/documents/:id"
      - regex: "path"
        action: labeldrop

    # 4. Hard backstop: refuse to ingest a target that misbehaves
    sample_limit: 20000
    label_limit: 30
    label_value_length_limit: 256
```

Naming conventions that survive handover

- [ ] `snake_case`, lowercase, no dots or dashes in Prometheus names.
- [ ] Prefix with the subsystem or application: `doc_`, `ingest_`, `llm_`.
- [ ] Base unit in the name and in base units: `_seconds` not `_ms`, `_bytes` not `_mb`, `_ratio` for 0–1 fractions. Convert for display, store in base units.
- [ ] Counters end in `_total`. Prometheus client libraries append it for you; do not double it.
- [ ] Use the same label names everywhere: `service`, `env`, `region`, `outcome`, `upstream`, `error_code`. Inconsistent label names make cross-service queries impossible.
- [ ] A metric name should read as a sentence with its unit: `doc_process_duration_seconds`, `llm_tokens_total`, `queue_oldest_message_age_seconds`.

- [ ] Never encode a value into the metric name: `requests_from_acme_total` is a label, not a name.

RED, USE and the four golden signals

Three overlapping frameworks, each useful for a different thing. Learn all three, because different customers will have standardised on different ones and you need to speak their language.

RED — FOR REQUEST-DRIVEN SERVICES

Rate (requests/sec), **E**rrors (failed requests/sec), **D**uration (latency distribution). Apply per endpoint, per service. This is the default for anything with an API.

USE — FOR RESOURCES

Utilisation (% time busy), **S**aturation (queued work), **E**rrors. Apply to CPU, memory, disk, network, connection pools, thread pools. Saturation is the leading indicator — utilisation at 80% with a growing queue is about to become an outage.

FOUR GOLDEN SIGNALS — GOOGLE SRE

Latency, traffic, errors, saturation. Essentially RED plus saturation. The key refinement: measure latency of successful and failed requests *separately*, because fast failures make your latency chart look great during an outage.

TEXT

```
APPLY THEM TOGETHER — a worked example for one service

  Service: doc-ingest (HTTP API + background workers + Postgres + SharePoint + LLM)

  RED on the API surface
    rate      sum(rate(http_requests_total[5m])) by (route)
    errors    sum(rate(http_requests_total{status=~"5.."}[5m])) by (route)
    duration  histogram_quantile(0.99, sum by (le,route) (
                rate(http_request_duration_seconds_bucket[5m])))

  RED on each dependency (this is the part people skip, and it is the valuable part)
    rate      sum(rate(upstream_calls_total[5m])) by (upstream)
    errors    sum(rate(upstream_calls_total{outcome="error"}[5m])) by (upstream)
    duration  histogram_quantile(0.95, sum by (le,upstream) (
                rate(upstream_duration_seconds_bucket[5m])))

  USE on the resources
    utilisation  rate(process_cpu_seconds_total[5m])
                 pg_stat_activity_count / pg_settings_max_connections
    saturation   ingest_queue_depth
                 node_load5 / machine_cpu_cores
                 go_goroutines  /  python threads blocked
    errors       node_network_receive_errs_total, pg_stat_database_deadlocks
```

Averages hide everything

If 99 requests take 50ms and one takes 10 seconds, the mean is 149ms — a number that describes no actual request and triggers no alert. Meanwhile one user waited ten seconds. Always look at the distribution. Always alert on p95 or p99, never on mean. And remember that p99 is not "the worst case": at 1,000 rps, p99 means 10 requests per second are worse than that, which is 864,000 unhappy requests a day.

Exemplars — the bridge from a metric to a trace

Exemplars attach a trace ID to a specific histogram bucket observation. You see a spike in the p99 panel, click the dot on the chart, and land on an actual trace of a request that was in that bucket. This is the single highest-value integration between metrics and traces and it is still under-used. It requires the OpenMetrics exposition format and a Prometheus started with `--enable-feature=exemplar-storage`.

PYTHON

```python
from prometheus_client import Histogram
from opentelemetry import trace

H = Histogram("http_request_duration_seconds", "latency", ["route", "status"])

def record(route, status, seconds):
    ctx = trace.get_current_span().get_span_context()
    exemplar = {"trace_id": format(ctx.trace_id, "032x")} if ctx.is_valid else None
    H.labels(route, status).observe(seconds, exemplar=exemplar)

# Exposition (OpenMetrics) then looks like:
#   http_request_duration_seconds_bucket{route="/v1/docs",le="2.0"} 4812 # {trace_id="
```

Prometheus in Practice

Prometheus is the de facto standard for metrics in anything Kubernetes-adjacent, and even customers who have bought Datadog or Dynatrace usually have a Prometheus somewhere. It is a pull-based time-series database with its own query language. Learn it properly: the exposition format is trivially easy to emit from any language, the data model is small enough to hold in your head, and PromQL fluency makes you dramatically faster during an incident than someone clicking through a UI.

The exposition format

Your application exposes a plain-text HTTP endpoint, usually `/metrics` on a separate port. Prometheus scrapes it every 15–60 seconds. That is the whole protocol. No agent, no SDK lock-in, no network egress from your process.

TEXT

```
# HELP http_requests_total Total HTTP requests served.
# TYPE http_requests_total counter
http_requests_total{route="/v1/documents",method="POST",status="200"} 184412
http_requests_total{route="/v1/documents",method="POST",status="500"} 37

# HELP http_request_duration_seconds Request latency.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{route="/v1/documents",le="0.1"}  12043
http_request_duration_seconds_bucket{route="/v1/documents",le="0.5"} 158221
http_request_duration_seconds_bucket{route="/v1/documents",le="1.0"} 181902
http_request_duration_seconds_bucket{route="/v1/documents",le="2.0"} 184100
http_request_duration_seconds_bucket{route="/v1/documents",le="+Inf"} 184449
http_request_duration_seconds_sum{route="/v1/documents"}   79221.4
http_request_duration_seconds_count{route="/v1/documents"} 184449

# HELP ingest_queue_depth Messages awaiting processing.
# TYPE ingest_queue_depth gauge
ingest_queue_depth{queue="ocr"} 42

# HELP build_info Build metadata. Value is always 1; the labels are the payload.
# TYPE build_info gauge
build_info{version="2.7.1",revision="a91f3c2",go_version="1.22.1"} 1
```

The info-metric pattern

`build_info{version="…"} 1` looks pointless but is the standard way to attach metadata without polluting every metric with a `version` label. You join it at query time: `sum(rate(http_requests_total[5m])) * on(instance) group_left(version) build_info`. This gives you "error rate by version" during a canary rollout without multiplying your series count by the number of versions you have ever shipped.

Scrape configuration that works in a customer's cluster

YAML

```yaml
global:
  scrape_interval: 30s          # 15s if you can afford it; 60s in constrained envs
  scrape_timeout: 10s           # must be < scrape_interval
  evaluation_interval: 30s
  external_labels:              # stamped on everything leaving this Prometheus
    cluster: acme-prod-eu
    region: eu-central-1
    vendor_deployment: fde-docpipeline   # so the customer can bill/attribute you

scrape_configs:
  # 1. Kubernetes pods that opt in with annotations
  - job_name: kubernetes-pods
    kubernetes_sd_configs: [{ role: pod }]
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: "true"
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
        action: replace
        target_label: __metrics_path__
        regex: (.+)
      - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_po
        action: replace
        regex: ([^:]+)(?::\d+)?;(\d+)
        replacement: $1:$2
        target_label: __address__
      - source_labels: [__meta_kubernetes_namespace]
        target_label: namespace
      - source_labels: [__meta_kubernetes_pod_label_app_kubernetes_io_name]
        target_label: service
      # Do NOT keep pod name as a label on high-churn deployments unless you need it
      - source_labels: [__meta_kubernetes_pod_name]
        target_label: pod

  # 2. A static target behind the customer's proxy, with mTLS
  - job_name: legacy-erp-exporter
    scheme: https
    static_configs:
      - targets: ["erp-exporter.internal.acme.corp:9113"]
    tls_config:
      ca_file:   /etc/prom/certs/acme-internal-ca.pem   # their private CA
      cert_file: /etc/prom/certs/prom-client.pem
      key_file:  /etc/prom/certs/prom-client.key
      server_name: erp-exporter.internal.acme.corp      # SNI must match their SAN
    proxy_url: "http://proxy.acme.corp:8080"

  # 3. Scrape through basic auth read from a file (never inline a secret)
  - job_name: rabbitmq
```

```
    static_configs: [{ targets: ["rabbit-exporter:9419"] }]
    basic_auth:
      username: prometheus
      password_file: /etc/prom/secrets/rabbit_password
```

⚠ **Pull does not work through a one-way firewall**

Prometheus must be able to reach your pods. In a customer environment where the monitoring cluster is in a different security zone, that is a firewall request that may take weeks, or be refused. Three escapes, in order of preference: (1) run a Prometheus inside the same zone and `remote_write` outward (outbound is usually allowed where inbound is not); (2) run an OpenTelemetry Collector in agent mode that scrapes locally and pushes; (3) as a last resort, Pushgateway — but understand that Pushgateway breaks the up/down semantics of scraping and is intended only for batch jobs that do not live long enough to be scraped.

PromQL — the subset you actually need

TEXT

```
SELECTORS
  http_requests_total                              all series with this name
  http_requests_total{service="doc-ingest"}        exact match
  http_requests_total{status=~"5.."}               regex match (anchored)
  http_requests_total{status!~"2..|3.."}           negative regex
  http_requests_total{service="doc-ingest"}[5m]    RANGE vector: raw samples, 5 min

RATE FAMILY  (counters only - these handle resets correctly)
  rate(x_total[5m])        per-second average over the window      ← use for alerting
  irate(x_total[5m])       per-second based on last TWO samples    ← spiky, use for f
  increase(x_total[1h])    total increase over window (= rate * window) ← use for "ho

  RULE OF THUMB: the range must be at least 4x the scrape interval so the window
  always contains ≥ 4 samples. 30s scrape → use [2m] minimum, [5m] is safe.

AGGREGATION  (the 'by' / 'without' clause controls the output labels)
  sum(rate(http_requests_total[5m]))                       one number, whole fleet
  sum by (route) (rate(http_requests_total[5m]))           one series per route
  sum without (pod, instance) (rate( ... ))                keep everything EXCEPT thes
  topk(5, sum by (route) (rate(http_requests_total[5m])))  the five busiest routes
  count(up == 0)                                           how many targets are down

LATENCY FROM HISTOGRAMS  (the le label MUST be preserved inside the sum)
  histogram_quantile(0.99,
    sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))

  Average latency (cheaper, less useful):
    rate(http_request_duration_seconds_sum[5m])
  / rate(http_request_duration_seconds_count[5m])

ERROR RATIO  (the canonical SLI expression)
    sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))

  Guard against divide-by-zero when there is no traffic:
     ... / clamp_min(sum(rate(http_requests_total[5m])), 0.001)

PREDICTION AND TREND
  predict_linear(node_filesystem_avail_bytes[6h], 4*3600) < 0   disk full in 4h
  deriv(ingest_queue_depth[30m]) > 0                            queue is growing
  changes(process_start_time_seconds[1h]) > 3                   crash-looping

TIME SHIFT AND COMPARISON
  sum(rate(http_requests_total[5m])) offset 1w                  same time last week
  sum(rate(http_requests_total[5m]))
    / sum(rate(http_requests_total[5m] offset 1w))              week-over-week ratio

VECTOR MATCHING  (joins - the part everyone finds hard)
```

```
# one-to-one: same label set on both sides
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes

# many-to-one with group_left: pull labels from the right-hand side
sum by (instance) (rate(http_requests_total[5m]))
  * on(instance) group_left(version, team) build_info

# ignoring labels that differ
sum by (service) (rate(errors_total[5m]))
  / ignoring(status) sum by (service) (rate(requests_total[5m]))

# unless: series on the left that have NO match on the right
up == 1 unless on(instance) node_boot_time_seconds
```

The three PromQL mistakes everyone makes once

1. `sum(rate(x[5m]))` not `rate(sum(x)[5m])` — you must take the rate *before* aggregating, because summing counters across instances that restart produces fake drops. **2.** Dropping the `le` label before `histogram_quantile` — it needs the buckets, so it must be `sum by (le, ...)`. **3.** Using a range shorter than the scrape interval — `rate(x[30s])` with a 30s scrape gives you one sample and therefore no data, and the panel silently shows nothing rather than erroring.

Worked queries for real questions

TEXT

```
Q: "Which tenant tier is seeing errors right now?"
   sum by (tenant_tier) (rate(http_requests_total{status=~"5.."}[5m]))
     / sum by (tenant_tier) (rate(http_requests_total[5m]))

Q: "Is the new version worse than the old one?" (during a canary)
   sum by (version) (rate(http_requests_total{status=~"5.."}[5m]))
     / sum by (version) (rate(http_requests_total[5m]))

Q: "How much of our latency is SharePoint?"
     histogram_quantile(0.95, sum by (le) (rate(upstream_duration_seconds_bucket{upstr
   / histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5

Q: "Are we about to hit the LLM rate limit?"
   sum(rate(llm_tokens_total[1m])) * 60          # tokens per minute
     / on() group_left llm_tokens_per_minute_quota

Q: "Which pods restarted in the last hour, and how often?"
   topk(10, increase(kube_pod_container_status_restarts_total[1h]) > 0)

Q: "Is the queue draining or growing?"   (positive = growing)
   deriv(ingest_queue_depth[15m])
   # and how long until it clears at the current drain rate:
   ingest_queue_depth / clamp_min(rate(ingest_processed_total[15m]), 0.001)

Q: "Certificates expiring in the next 30 days"
   (tls_cert_expiry_timestamp_seconds - time()) / 86400 < 30

Q: "Connection pool saturation"
   pgbouncer_pools_server_active_connections
     / pgbouncer_pools_server_max_connections > 0.8

Q: "Apdex-style score with a 1s target and 4s tolerable threshold"
   (  sum(rate(http_request_duration_seconds_bucket{le="1"}[5m]))
    + sum(rate(http_request_duration_seconds_bucket{le="4"}[5m])) ) / 2
   / sum(rate(http_request_duration_seconds_count[5m]))

Q: "Memory trend — are we leaking?"
   deriv(process_resident_memory_bytes[6h]) > 0
   and
   process_resident_memory_bytes > 0.8 * container_spec_memory_limit_bytes
```

Recording rules: precompute the expensive things

A query that scans 50,000 series over 30 days will time out on a dashboard and will definitely time out when seven people open the dashboard during an incident. Recording rules evaluate an expression on a

schedule and store the result as a new series. Use them for anything that appears on more than one dashboard, anything an alert depends on, and anything that a human types more than twice.

YAML

```yaml
groups:
  - name: doc-ingest.sli
    interval: 30s
    rules:
      # Naming convention: level:metric:operation
      - record: service:http_requests:rate5m
        expr: sum by (service, route, status_class) (
                label_replace(rate(http_requests_total[5m]),
                              "status_class", "$1", "status", "(.).."))

      - record: service:http_request_errors:ratio_rate5m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
            /
          clamp_min(sum by (service) (rate(http_requests_total[5m])), 0.001)

      # Pre-compute every burn-rate window the alerts need
      - record: service:sli_error:ratio_rate5m
        expr: sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
                / clamp_min(sum by (service) (rate(http_requests_total[5m])), 0.001)
      - record: service:sli_error:ratio_rate30m
        expr: sum by (service) (rate(http_requests_total{status=~"5.."}[30m]))
                / clamp_min(sum by (service) (rate(http_requests_total[30m])), 0.001)
      - record: service:sli_error:ratio_rate1h
        expr: sum by (service) (rate(http_requests_total{status=~"5.."}[1h]))
                / clamp_min(sum by (service) (rate(http_requests_total[1h])), 0.001)
      - record: service:sli_error:ratio_rate6h
        expr: sum by (service) (rate(http_requests_total{status=~"5.."}[6h]))
                / clamp_min(sum by (service) (rate(http_requests_total[6h])), 0.001)

      - record: service:latency_p99:5m
        expr: histogram_quantile(0.99, sum by (le, service, route) (
                rate(http_request_duration_seconds_bucket[5m])))
```

Alertmanager: routing, grouping and silences

YAML

```yaml
route:
  receiver: default
  group_by: ['alertname', 'cluster', 'service']   # one notification per group
  group_wait: 30s          # wait for siblings before first notify
  group_interval: 5m       # wait before notifying about NEW alerts in an existing gro
  repeat_interval: 4h      # re-notify about unresolved alerts
  routes:
    - matchers: ['severity = critical']
      receiver: pagerduty
      group_wait: 10s
      repeat_interval: 1h
      continue: true                # ALSO fall through to the chat receiver
    - matchers: ['severity = critical']
      receiver: customer-teams-channel
    - matchers: ['severity = warning']
      receiver: customer-teams-channel
      repeat_interval: 12h
    - matchers: ['severity = info']
      receiver: "null"              # record it, never notify

inhibit_rules:
  # If the whole service is down, do not also page about its latency
  - source_matchers: ['alertname = ServiceDown']
    target_matchers: ['alertname =~ "High.*|Slow.*"']
    equal: ['service', 'cluster']
  # Critical suppresses warning for the same alert and service
  - source_matchers: ['severity = critical']
    target_matchers: ['severity = warning']
    equal: ['alertname', 'service', 'cluster']

receivers:
  - name: default
  - name: "null"
  - name: pagerduty
    pagerduty_configs:
      - routing_key_file: /etc/alertmanager/secrets/pd_key
        description: '{{ .CommonLabels.alertname }} on {{ .CommonLabels.service }}'
        details:
          runbook:  '{{ .CommonAnnotations.runbook_url }}'
          dashboard:'{{ .CommonAnnotations.dashboard_url }}'
          summary:  '{{ .CommonAnnotations.summary }}'
  - name: customer-teams-channel
    msteams_configs:
      - webhook_url_file: /etc/alertmanager/secrets/teams_webhook
        title: '[{{ .Status | toUpper }}] {{ .CommonLabels.alertname }}'
        text: |
          {{ range .Alerts }}*{{ .Annotations.summary }}*
          Service: {{ .Labels.service }} / {{ .Labels.cluster }}
```

```
        Runbook: {{ .Annotations.runbook_url }}
        {{ end }}
```

> ✦ **Route to the customer, not to you**
> You will not be on call for this system in four months. Configure Alertmanager to deliver to the customer's existing channel — their PagerDuty service, their ServiceNow queue, their Teams channel — from day one, with you merely CC'd. If alerts go to your inbox during the engagement and are redirected at handover, they will be redirected badly and silently. Make the customer's team the primary recipient while you are still there to fix the routing.

Exporters: instrumenting what you cannot change

You will constantly need metrics from things you did not write: the customer's Oracle database, their F5 load balancer, an IBM MQ broker, a Windows service. Exporters are small processes that translate some system's native interface into the Prometheus exposition format.

Target	Exporter	Key metrics to pull
Linux host	`node_exporter`	CPU, memory, disk, filesystem, network, load, systemd units
Windows host	`windows_exporter`	CPU, memory, disk, IIS, services, scheduled tasks
Containers	cAdvisor / kubelet	Per-container CPU throttling, memory working set, OOM kills
PostgreSQL	`postgres_exporter`	Connections, locks, replication lag, slow queries, bloat
Oracle / SQL Server	`oracledb_exporter`, `sql_exporter`	Sessions, wait events, tablespace free, custom SQL
RabbitMQ / Kafka	built-in / `kafka_exporter`	Queue depth, consumer lag, unacked, partitions offline
Any HTTP/TCP endpoint	`blackbox_exporter`	Reachability, status code, TLS expiry, response time, body regex
Arbitrary scripts	`textfile` collector	Anything a cron job can write to a `.prom` file

YAML

```yaml
# blackbox_exporter: the single most useful exporter for an FDE.
# It proves reachability FROM INSIDE the customer's network, which is the thing
# you can never test from your laptop.
modules:
  http_2xx_with_tls:
    prober: http
    timeout: 10s
    http:
      valid_status_codes: [200, 204]
      method: GET
      preferred_ip_protocol: ip4
      fail_if_body_not_matches_regexp: ['"status"\s*:\s*"ok"']
      tls_config:
        ca_file: /etc/ssl/acme-internal-ca.pem   # the customer's private CA
  tcp_connect:
    prober: tcp
    timeout: 5s
  sap_rfc_port:
    prober: tcp
    timeout: 5s

---

# And the scrape job that uses it
scrape_configs:
  - job_name: blackbox
    metrics_path: /probe
    params: { module: [http_2xx_with_tls] }
    static_configs:
      - targets:
          - https://sharepoint.acme.corp/_api/web
          - https://sap-gw.acme.corp:44300/sap/opu/odata/
          - https://login.microsoftonline.com/acme.onmicrosoft.com/v2.0/.well-known/op
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: target
      - target_label: __address__
        replacement: blackbox-exporter:9115

# Then alert on:
#   probe_success == 0
#   probe_ssl_earliest_cert_expiry - time() < 14 * 86400
#   probe_duration_seconds > 5
```

✓ **Why blackbox matters so much on site**

When a customer says "your service cannot reach SAP", you need to prove whether the problem is your code, their network, or SAP itself. A blackbox probe running continuously inside their network gives you a chart that answers this without argument. It takes twenty minutes to deploy and it has settled more finger-pointing meetings than any other tool I know of. Deploy it in week one, before you need it, so you have baseline history.

Grafana Dashboards People Actually Use

Most dashboards are built for the person building them and abandoned within a month. The dashboard you leave behind at a customer has a brutal test applied to it: an L1 operator who has never met you, at 2am, under pressure, must be able to open it and decide within thirty seconds whether to escalate. Design for that person, not for yourself.

The one-screen-per-service rule

Every service gets exactly one dashboard that fits on one 1080p screen without scrolling. If it does not fit, you have too many panels, not too small a screen. The layout is fixed and identical across every service you deploy, so that muscle memory transfers:

TEXT

```
ROW 1 — IS IT UP AND IS IT HEALTHY?   (4 stat panels, big numbers, colour-coded)
  [ Availability 99.94% ]  [ Error rate 0.06% ]  [ p99 latency 1.8s ]  [ RPS 412 ]
  Each has a threshold: green / amber / red. No sparklines, no axes. Just the number.

ROW 2 — THE GOLDEN SIGNALS OVER TIME  (3 time series, 6h default window)
  [ Requests/sec by status ]  [ Latency p50/p95/p99 ]  [ Errors/sec by error_code ]
  Deploy annotations overlaid on all three. This is where you spot "it started at 14:0

ROW 3 — DEPENDENCIES                  (1 table + 1 time series)
  [ Upstream health table: system | success% | p95 | last error ]
  [ Upstream latency p95 by system — the finger-pointing resolver ]

ROW 4 — SATURATION AND CAPACITY       (4 small panels)
  [ Queue depth ]  [ CPU vs limit ]  [ Memory vs limit ]  [ Connection pool used/max ]

ROW 5 — LINKS (a text panel, not a graph)
  Runbook | Logs for this service | Traces for this service | On-call rota | Escalatio
```

> ⚠ **Panels you should delete**
>
> Any panel showing a raw counter that only goes up. Any pie chart. Any panel whose title is a metric name rather than a question. Any panel with more than eight series on it (nobody can read a legend with forty entries). Any panel nobody has looked at in ninety days — Grafana Enterprise will tell you this; in OSS, ask the team. Dashboards rot, and a dashboard full of dead panels teaches operators to ignore dashboards.

Template variables: one dashboard, every environment

JSON

```
{
  "templating": {
    "list": [
      { "name": "cluster", "type": "query", "datasource": "Prometheus",
        "query": "label_values(up, cluster)", "refresh": 1, "includeAll": false },

      { "name": "namespace", "type": "query", "datasource": "Prometheus",
        "query": "label_values(up{cluster=\"$cluster\"}, namespace)", "refresh": 2 },

      { "name": "service", "type": "query", "datasource": "Prometheus",
        "query": "label_values(http_requests_total{cluster=\"$cluster\",namespace=\"$n
        "refresh": 2, "multi": true, "includeAll": true },

      { "name": "quantile", "type": "custom",
        "query": "0.50,0.90,0.95,0.99", "current": { "value": "0.99" } },

      { "name": "interval", "type": "interval",
        "query": "1m,5m,15m,1h", "auto": true, "auto_count": 50 }
    ]
  }
}
```

Then every panel query uses `$cluster`, `$namespace`, `$service`, `$__rate_interval`. One dashboard JSON covers dev, staging and six customer production clusters. Ship it as a provisioned file in the Helm chart, not as something clicked together in the UI — a dashboard that exists only in someone's Grafana database is lost at the next upgrade.

YAML

```yaml
# Provisioning: dashboards as code, shipped with the application
# /etc/grafana/provisioning/dashboards/fde.yaml
apiVersion: 1
providers:
  - name: fde-docpipeline
    orgId: 1
    folder: "Vendor / DocPipeline"
    type: file
    disableDeletion: true        # stop an operator deleting it by accident
    allowUiUpdates: false        # changes must come through git
    updateIntervalSeconds: 60
    options:
      path: /var/lib/grafana/dashboards/fde

---

# In Kubernetes, the Grafana sidecar picks up labelled ConfigMaps automatically
apiVersion: v1
kind: ConfigMap
metadata:
  name: docpipeline-dashboard
  labels:
    grafana_dashboard: "1"       # the sidecar watches for this label
data:
  docpipeline.json: |
    { "title": "DocPipeline / Service Overview", "uid": "fde-docpipe-svc", ... }
```

Always use $__rate_interval

Grafana's `$__rate_interval` variable computes a range that is at least four scrape intervals and scales with the zoom level. Hard-coding `[5m]` means your panel breaks when someone zooms out to 30 days (too few points) or zooms in to 5 minutes (no data). Write `rate(http_requests_total[$__rate_interval])` and it is correct at every zoom level. This single habit fixes most "the dashboard looks wrong" complaints.

Annotations: the change feed on every chart

Overlaying deploys on a latency chart converts "latency went up" into "latency went up when you deployed 2.7.1" in zero seconds. This is the highest value-per-line-of-config item in all of Grafana.

JSON

```
{
  "annotations": {
    "list": [
      {
        "name": "Deploys",
        "datasource": "Prometheus",
        "enable": true,
        "iconColor": "rgba(37, 99, 235, 1)",
        "expr": "changes(build_info[$__rate_interval]) > 0",
        "titleFormat": "Deploy {{version}}",
        "textFormat": "{{service}} → {{version}} ({{revision}})"
      },
      {
        "name": "Alerts firing",
        "datasource": "Prometheus",
        "enable": true,
        "iconColor": "red",
        "expr": "ALERTS{alertstate=\"firing\", severity=\"critical\"}",
        "titleFormat": "{{alertname}}"
      },
      {
        "name": "Customer change window",
        "datasource": "-- Grafana --",
        "enable": true,
        "iconColor": "orange",
        "tags": ["change-window"]
      }
    ]
  }
}
```

✓ **Ask for their change calendar**

The customer's infrastructure team patches on a schedule you were not told about. Getting their change-window annotations onto your dashboard — even manually, via the Grafana annotations API in a weekly script — means that when your service degrades at 02:00 on the third Sunday, you can see the words "ACME: monthly Windows patching" on the chart instead of spending four hours bisecting your own code.

BASH

```
# Push an annotation from a deploy pipeline — two lines in your CD job
curl -sS -X POST "$GRAFANA_URL/api/annotations" \
  -H "Authorization: Bearer $GRAFANA_TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"time\": $(date +%s000),
       \"tags\": [\"deploy\",\"docpipeline\",\"$ENVIRONMENT\"],
       \"text\": \"Deployed docpipeline $VERSION ($GIT_SHA) by $DEPLOYER\"}"
```

Panel-level craft

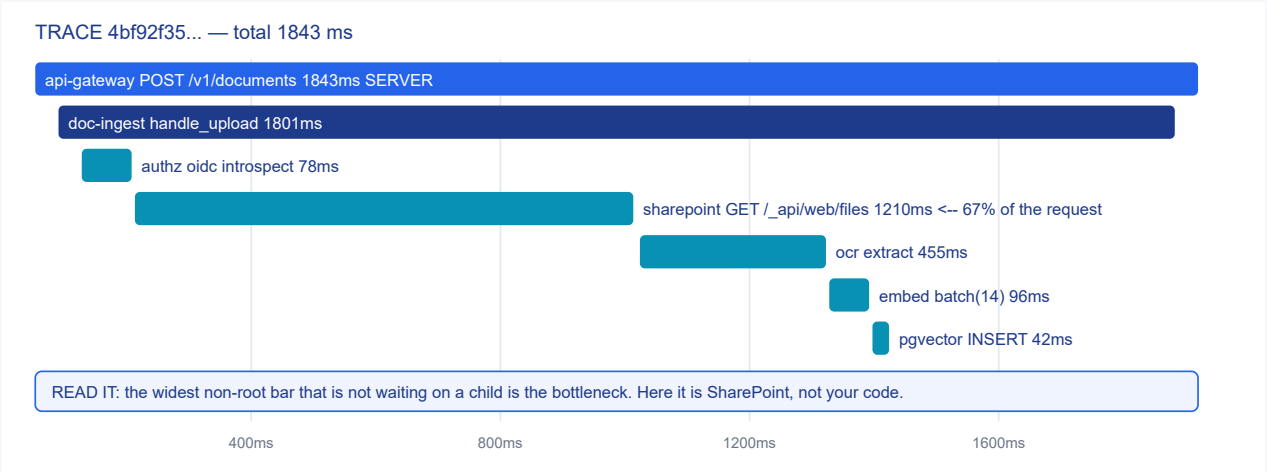
- [ ] Title every panel as a **question**: "Are requests failing?" beats "http_requests_total".
- [ ] Set units on every axis. A y-axis reading "0.0184" when it means 1.84% is a misread waiting to happen. Use `percentunit` for 0–1 ratios.
- [ ] Put thresholds on stat panels so colour does the work: green under SLO, amber within 20% of it, red over.
- [ ] Use the panel **description** field (the little `i` in the corner) to explain what the metric is and what to do when it is bad. This is where the runbook summary lives.
- [ ] Add **data links** from panels to the matching log query and trace search, pre-filtered by the current variables and time range. Clicking from a spike to the logs of that spike should be one click.
- [ ] Set a sensible default time range — 6 hours for a service overview, 30 days for a capacity dashboard.
- [ ] Use `Last` + `Max` in legends, not `Avg`, for latency panels.
- [ ] Pin the colour of `status=5xx` to red and `2xx` to green with field overrides, consistently, on every dashboard.

Distributed Tracing

A trace is the record of one request's journey through a distributed system, represented as a tree of spans. It answers the question that logs and metrics structurally cannot: *where did the time go, and what called what?* In an enterprise deployment where a single user action touches your API, the customer's identity provider, their SharePoint, a vector database and an LLM endpoint, a trace is the only artefact that shows the whole path in one picture.

The data model

Concept	What it is	Example
Trace	The whole request. Identified by a 128-bit trace ID (32 hex chars).	`4bf92f3577b34da6a3ce929d0e0e4736`
Span	One unit of work: a name, start time, duration, status, parent. 64-bit ID.	`POST /v1/documents`, 1843ms
Parent/child	Causality. A child span was caused by its parent.	HTTP handler → DB query
Attributes	Key/value pairs on a span. High cardinality is fine here.	`tenant.id=acme`, `doc.pages=14`
Events	Timestamped points inside a span.	`exception`, `retry.attempt`, `cache.evicted`
Links	Reference to a span in another trace.	A batch job span linking to the 500 messages it processed
Span kind	SERVER, CLIENT, PRODUCER, CONSUMER, INTERNAL	Determines how backends compute service graphs
Resource	Attributes of the emitting process, identical on all its spans.	`service.name`, `service.version`, `k8s.pod.name`



A flame graph of one trace. Width is duration; nesting is causality.

Context propagation and W3C traceparent

Propagation is what makes a trace distributed. The parent service serialises its span context into an HTTP header; the child deserialises it and makes its spans children of that context. The W3C Trace Context standard defines two headers, and essentially every vendor now supports them.

TEXT

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             |  |                                |                |
             |  |                                |                +-- trace-flags (01
             |  |                                +------------------- parent-span-id (
             |  +-------------------------------------------------- trace-id (32 hex
             +----------------------------------------------------- version (00)

tracestate: acme=t61rcWkgMzE,vendorb=00f067aa0ba902b7
            vendor-specific key/value list, max 32 entries, preserved across hops

LEGACY FORMATS YOU WILL STILL MEET IN ENTERPRISES
  B3 multi-header (Zipkin/Istio):  X-B3-TraceId, X-B3-SpanId, X-B3-ParentSpanId, X-B3-
  B3 single:                       b3: {traceId}-{spanId}-{samplingState}-{parentSpanI
  Jaeger:                          uber-trace-id: {trace-id}:{span-id}:{parent-span-id
  AWS X-Ray:                       X-Amzn-Trace-Id: Root=1-5759e988-bd862e3f ... ;Sample
  Datadog:                         x-datadog-trace-id, x-datadog-parent-id, x-datadog-

  Configure OTEL to accept all of them, because the customer's service mesh,
  API gateway and legacy Java services each speak a different one:
    OTEL_PROPAGATORS=tracecontext,baggage,b3multi,jaeger,xray
```

⚠ **Where propagation silently breaks**

Message queues (headers are not carried unless you explicitly put them in the message), thread pools and background executors (context is thread-local or async-local and does not cross a `submit()` boundary), `setTimeout`/scheduled jobs, batch processing, third-party SDKs that construct their own HTTP client, and anything going through an API gateway configured to strip unknown headers. The symptom is a trace that ends abruptly at a service boundary, or a flood of single-span "traces". Test propagation explicitly as part of your deployment verification.

PYTHON

```python
# Propagating context through a queue — the case everyone forgets
from opentelemetry import trace, propagate
from opentelemetry.trace import SpanKind

tracer = trace.get_tracer(__name__)

def publish(queue, payload: dict):
    with tracer.start_as_current_span("doc.enqueue", kind=SpanKind.PRODUCER) as span:
        span.set_attribute("messaging.system", "rabbitmq")
        span.set_attribute("messaging.destination.name", queue)
        headers: dict[str, str] = {}
        propagate.inject(headers)                # writes traceparent + tracestate
        channel.basic_publish(
            exchange="", routing_key=queue,
            body=json.dumps(payload),
            properties=pika.BasicProperties(headers=headers),
        )

def on_message(channel, method, properties, body):
    ctx = propagate.extract(properties.headers or {})
    # CONSUMER span is a child of the producer's span, across the broker
    with tracer.start_as_current_span("doc.process", context=ctx,
                                      kind=SpanKind.CONSUMER) as span:
        span.set_attribute("messaging.message.body.size", len(body))
        span.set_attribute("messaging.rabbitmq.delivery_tag", method.delivery_tag)
        try:
            process(json.loads(body))
            channel.basic_ack(method.delivery_tag)
        except Exception as e:
            span.record_exception(e)
            span.set_status(trace.Status(trace.StatusCode.ERROR, str(e)))
            channel.basic_nack(method.delivery_tag, requeue=False)   # → DLQ
            raise
```

PYTHON

```python
# Propagating across a thread pool — context does NOT follow a submit()
from concurrent.futures import ThreadPoolExecutor
from opentelemetry import context as otel_context

pool = ThreadPoolExecutor(max_workers=8)

def submit_with_context(fn, *args, **kwargs):
    carrier = otel_context.get_current()          # capture on the calling thread
    def wrapper():
        token = otel_context.attach(carrier)      # restore on the worker thread
        try:
            return fn(*args, **kwargs)
        finally:
            otel_context.detach(token)
    return pool.submit(wrapper)
```

Sampling strategies

You cannot keep every trace at scale, and you should not try. The question is which traces to keep and who decides.

Strategy	Decision made	Pros	Cons
Always on	Keep everything	Nothing is missing	Only viable under ~50 rps or in dev
Head-based probabilistic	At the root span, before anything happens	Cheap, simple, consistent across services	Throws away errors and slow requests at the same rate as boring ones
Parent-based	Honour upstream's decision	Guarantees complete traces	Downstream has no say; a chatty client can flood you
Tail-based	After the trace is complete, in the Collector	Keep all errors, all slow traces, a sample of the rest	Collector must buffer whole traces in memory; needs all spans of a trace to reach the same collector instance
Rate limiting	N traces/sec per service	Predictable cost ceiling	Under-samples low-traffic, high-value endpoints
Remote / adaptive	Backend pushes rates to agents	Tunable without redeploy — valuable when you cannot redeploy	More moving parts

✦ **The pragmatic default**

Head-based at 100% into a Collector, then **tail-based in the Collector**: keep 100% of traces containing an error, 100% of traces slower than your SLO, 100% of traces for a named debug tenant, and 1–5% of the rest. You pay the network cost of shipping all spans to the local Collector (cheap, it is on the same node) and the storage cost of only the interesting ones. This is the configuration I deploy by default and it has never needed defending to a customer's cost owner.

Reading a trace: what to look for

1. **Find the widest bar that has no wide children.**
   That is where time is actually being spent, as opposed to time spent waiting on something else. If a 1800ms parent has a 1700ms child, the parent is innocent.

2. **Look for gaps.**
   White space between a parent's start and its first child, or between siblings, is unaccounted time: queue wait, lock contention, GC pause, thread-pool starvation, or an uninstrumented call.

Gaps are where the interesting bugs live.

3. **Count repeated spans.**
Forty-one identical `SELECT` spans in a row is the N+1 query problem rendered visually. You will spot it in two seconds in a flame graph and never in a log.

4. **Check serial versus parallel.**
Six sibling spans arranged in a staircase are sequential calls that could have been concurrent. That is often a 5× latency win for an afternoon's work.

5. **Compare a fast trace with a slow one.**
Pull one p50 trace and one p99 trace for the same operation and diff them structurally. The difference is the cause, far more reliably than staring at the slow one alone.

6. **Read the span attributes on the slow span.**
`db.statement`, `http.url`, `retry.count`, `cache.hit`. The answer is usually literally written there.

OpenTelemetry End to End

OpenTelemetry is the CNCF standard for generating and shipping telemetry, and for a Forward Deployed Engineer it is not merely a nice option — it is the only sane choice. It is the one instrumentation API that lets you write your code once and ship it into a customer running Datadog, a second running Splunk, a third running Elastic, and a fourth running nothing but a Prometheus and a Loki, by changing a config file instead of your code. That property alone is worth the learning curve.



Auto-instrumentation versus manual

AUTO-INSTRUMENTATION GIVES YOU

HTTP server and client spans, database client spans with `db.statement`, message producer/consumer spans, gRPC, Redis, and framework-level context propagation — with zero code changes. In Java it is a `-javaagent` flag. In Python, `opentelemetry-instrument python app.py`. In Node, `--require ./otel.js`. In .NET, an environment variable. Start here, always.

MANUAL INSTRUMENTATION GIVES YOU

Business-meaningful spans (`doc.ocr`, `invoice.match`, `llm.generate`), business attributes (`tenant.id`, `doc.pages`, `llm.tokens`), and the context propagation that auto-instrumentation cannot infer (queues, thread pools, callbacks). This is where the value is, but it is worth much less without the auto layer underneath it.

BASH

```
# Zero-code: Python
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap --action=install     # installs instrumentation for what it fi

export OTEL_SERVICE_NAME=doc-ingest
export OTEL_RESOURCE_ATTRIBUTES="service.version=2.7.1,deployment.environment=prod,ser
export OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector.observability:4317
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
export OTEL_TRACES_SAMPLER=parentbased_traceidratio
export OTEL_TRACES_SAMPLER_ARG=1.0           # sample at the Collector instead
export OTEL_PROPAGATORS=tracecontext,baggage,b3multi
export OTEL_PYTHON_LOG_CORRELATION=true      # inject trace_id into stdlib logging
export OTEL_PYTHON_EXCLUDED_URLS="/health,/healthz,/metrics,/readyz"

opentelemetry-instrument python -m uvicorn app:main --host 0.0.0.0 --port 8080

# Zero-code: Java (the most mature agent by a distance)
java -javaagent:/opt/otel/opentelemetry-javaagent.jar \
     -Dotel.service.name=invoice-matcher \
     -Dotel.exporter.otlp.endpoint=http://otel-collector:4317 \
     -Dotel.instrumentation.jdbc.enabled=true \
     -Dotel.instrumentation.common.db-statement-sanitizer.enabled=true \
     -jar app.jar

# Zero-code: Node
node --require @opentelemetry/auto-instrumentations-node/register app.js

# Kubernetes: the OpenTelemetry Operator injects the agent for you
#   annotate the pod and the operator mutates it at admission time
#   instrumentation.opentelemetry.io/inject-java: "true"
```

PYTHON

```python
# Manual spans where they matter. Note what is an attribute vs an event.
from opentelemetry import trace, metrics, baggage
from opentelemetry.trace import SpanKind, Status, StatusCode

tracer = trace.get_tracer("doc-ingest", "2.7.1")
meter  = metrics.get_meter("doc-ingest", "2.7.1")

pages_hist = meter.create_histogram("doc.pages", unit="{page}",
                                    description="Pages per processed document")

def process(doc_id: str, tenant: str):
    with tracer.start_as_current_span("doc.process", kind=SpanKind.INTERNAL) as span:
        # Semantic-convention attributes where they exist, custom ones namespaced
        span.set_attribute("tenant.id", tenant)
        span.set_attribute("doc.id", doc_id)

        # baggage propagates to EVERY downstream service — use sparingly,
        # it rides on every HTTP request and can leak across trust boundaries
        ctx = baggage.set_baggage("tenant.id", tenant)

        try:
            with tracer.start_as_current_span("doc.fetch", kind=SpanKind.CLIENT) as s:
                s.set_attribute("peer.service", "sharepoint")
                s.set_attribute("server.address", "sharepoint.acme.corp")
                blob, meta = sharepoint.fetch(doc_id)
                s.set_attribute("http.response.body.size", len(blob))

            span.set_attribute("doc.pages", meta.pages)
            span.set_attribute("doc.mime", meta.mime)
            pages_hist.record(meta.pages, {"doc.mime": meta.mime, "tenant.tier": tier(

            with tracer.start_as_current_span("doc.ocr") as s:
                s.add_event("ocr.engine.selected", {"engine": "tesseract-5", "lang": "
                text = ocr.extract(blob)
                s.set_attribute("ocr.chars", len(text))
                if meta.pages > 50:
                    s.add_event("ocr.large_document", {"pages": meta.pages})

            span.set_status(Status(StatusCode.OK))
        except UpstreamTimeout as e:
            span.record_exception(e)                # adds an exception event + stack
            span.set_status(Status(StatusCode.ERROR, "sharepoint timeout"))
            span.set_attribute("error.type", "UpstreamTimeout")
            raise
```

Span attributes are not a dumping ground

Attributes are indexed by most backends, and some vendors bill per indexed attribute. More importantly: **span attributes leave your process and land in a system you do not control.** Never put a document body, a prompt containing customer data, an access token, or a raw SQL statement with inlined literal values into a span attribute. The OTEL Java agent's `db-statement-sanitizer` replaces literals with `?` for exactly this reason — make sure it is on.

The Collector — the component that earns its keep on site

The Collector is a standalone binary that receives telemetry, transforms it, and exports it somewhere. It is the single most useful piece of OpenTelemetry for an FDE, because it is the abstraction layer between your application and whatever the customer happens to own. You change the Collector config; the application never changes.

RECEIVERS

How data gets in: OTLP (gRPC/HTTP), Prometheus scrape, Jaeger, Zipkin, Fluent Forward, filelog, syslog, statsd, kafka, hostmetrics, k8s_events.

PROCESSORS

What happens in between: batch, memory_limiter, resource, attributes, filter, transform, tail_sampling, redaction, k8sattributes, probabilistic_sampler, span metrics.

EXPORTERS

Where it goes: OTLP, prometheusremotewrite, loki, elasticsearch, splunk_hec, datadog, azuremonitor, awsemf, kafka, file, debug.

YAML

```yaml
# ============================================================================
# otel-collector-config.yaml — a gateway config I have shipped to customers.
# The whole point: the app emits OTLP and knows nothing about the destination.
# ============================================================================
receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317, max_recv_msg_size_mib: 16 }
      http: { endpoint: 0.0.0.0:4318 }

  # Scrape Prometheus endpoints and convert them to OTLP metrics
  prometheus:
    config:
      scrape_configs:
        - job_name: otel-collector
          scrape_interval: 30s
          static_configs: [{ targets: ["127.0.0.1:8888"] }]

  # Tail container logs where the app cannot be changed to emit OTLP
  filelog:
    include: [/var/log/pods/*/*/*.log]
    include_file_path: true
    operators:
      - type: container
        id: parse-cri
      - type: json_parser
        parse_from: body
        if: 'body matches "^\\{"'

processors:
  # ALWAYS first in the chain. Prevents the Collector OOMing the node, which
  # in a customer cluster is a resume-generating event.
  memory_limiter:
    check_interval: 1s
    limit_percentage: 75
    spike_limit_percentage: 20

  # Enrich with Kubernetes metadata the app cannot know about itself
  k8sattributes:
    auth_type: serviceAccount
    passthrough: false
    extract:
      metadata: [k8s.namespace.name, k8s.pod.name, k8s.node.name,
                 k8s.deployment.name, k8s.pod.uid]
      labels:
        - { tag_name: app, key: app.kubernetes.io/name, from: pod }
        - { tag_name: team, key: owner, from: namespace }
```

```
# Stamp deployment identity so the customer can attribute cost and ownership
resource:
  attributes:
    - { key: deployment.environment, value: prod, action: upsert }
    - { key: vendor.deployment, value: fde-docpipeline, action: upsert }
    - { key: cloud.region, value: eu-central-1, action: upsert }

# Scrub PII before it ever reaches the customer's backend. Defence in depth.
attributes/scrub:
  actions:
    - { key: http.request.header.authorization, action: delete }
    - { key: http.request.header.cookie, action: delete }
    - { key: user.email, action: delete }
    - { key: db.statement, action: hash }
    - { key: enduser.id, action: hash }

transform/redact:
  error_mode: ignore
  trace_statements:
    - context: span
      statements:
        - replace_pattern(attributes["http.url"],
            "(email|token|key|secret)=[^&]*", "$$1=REDACTED")
        - replace_pattern(attributes["doc.filename"],
            "^.*$$", "[REDACTED]")

# Drop health-check noise: ~40% of span volume in a typical k8s deployment
filter/healthchecks:
  error_mode: ignore
  traces:
    span:
      - 'attributes["http.route"] == "/healthz"'
      - 'attributes["http.route"] == "/readyz"'
      - 'attributes["http.route"] == "/metrics"'
      - 'IsMatch(name, ".*kube-probe.*")'

# Derive RED metrics from spans. Gives you metrics for services that only
# emit traces — extremely useful when you cannot modify a legacy service.
spanmetrics:
  metrics_flush_interval: 30s
  histogram:
    explicit:
      buckets: [10ms, 50ms, 100ms, 250ms, 500ms, 1s, 2s, 5s, 10s, 30s]
  dimensions:
    - { name: http.route }
    - { name: http.response.status_code }
    - { name: tenant.tier }

# Tail sampling: keep what matters, drop the rest. Must run on a gateway
```

```
  # that receives ALL spans of a trace (use loadbalancing exporter upstream).
  tail_sampling:
    decision_wait: 30s          # must exceed your p99.9 trace duration
    num_traces: 100000
    expected_new_traces_per_sec: 2000
    policies:
      - name: keep-all-errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: keep-slow
        type: latency
        latency: { threshold_ms: 2000 }
      - name: keep-debug-tenant
        type: string_attribute
        string_attribute: { key: tenant.id, values: [acme-debug, pilot-group] }
      - name: keep-rate-limited
        type: numeric_attribute
        numeric_attribute: { key: http.response.status_code, min_value: 429, max_value
      - name: sample-the-rest
        type: probabilistic
        probabilistic: { sampling_percentage: 2 }

  batch:
    timeout: 5s
    send_batch_size: 8192
    send_batch_max_size: 16384

exporters:
  # Destination A: the customer already has Splunk
  splunk_hec:
    token: ${env:SPLUNK_HEC_TOKEN}
    endpoint: https://splunk-hec.acme.corp:8088/services/collector
    source: otel
    index: vendor_docpipeline
    tls: { ca_file: /etc/ssl/acme-internal-ca.pem }

  # Destination B: their Prometheus, via remote write (outbound = allowed)
  prometheusremotewrite:
    endpoint: https://mimir.acme.corp/api/v1/push
    tls: { ca_file: /etc/ssl/acme-internal-ca.pem }
    headers: { X-Scope-OrgID: vendor-docpipeline }
    resource_to_telemetry_conversion: { enabled: true }
    remote_write_queue: { enabled: true, num_consumers: 5, queue_size: 10000 }

  # Destination C: traces to their Tempo/Jaeger
  otlp/tempo:
    endpoint: tempo-distributor.observability:4317
    tls: { insecure: false, ca_file: /etc/ssl/acme-internal-ca.pem }
    sending_queue: { enabled: true, num_consumers: 10, queue_size: 5000 }
```

```
    retry_on_failure: { enabled: true, initial_interval: 5s, max_interval: 30s,
                        max_elapsed_time: 300s }

  # Destination D: air-gapped fallback — write to disk for manual export
  file/archive:
    path: /var/otel/archive/telemetry.jsonl
    rotation: { max_megabytes: 100, max_days: 7, max_backups: 10 }

extensions:
  health_check: { endpoint: 0.0.0.0:13133 }
  pprof:        { endpoint: 127.0.0.1:1777 }
  zpages:       { endpoint: 127.0.0.1:55679 }   # live debug UI for the Collector itse

service:
  extensions: [health_check, pprof, zpages]
  telemetry:
    logs:    { level: info }
    metrics: { level: detailed, address: 0.0.0.0:8888 }
  pipelines:
    traces:
      receivers:  [otlp]
      processors: [memory_limiter, k8sattributes, resource, attributes/scrub,
                   transform/redact, filter/healthchecks, spanmetrics,
                   tail_sampling, batch]
      exporters:  [otlp/tempo]
    metrics:
      receivers:  [otlp, prometheus, spanmetrics]
      processors: [memory_limiter, k8sattributes, resource, batch]
      exporters:  [prometheusremotewrite]
    logs:
      receivers:  [otlp, filelog]
      processors: [memory_limiter, k8sattributes, resource, attributes/scrub, batch]
      exporters:  [splunk_hec, file/archive]
```

Agent plus gateway

The standard topology is a Collector **agent** as a DaemonSet on every node (cheap, local, adds node/pod metadata, handles the app's OTLP with near-zero network cost) feeding a Collector **gateway** deployment of 2–5 replicas (does the expensive work: tail sampling, enrichment, export, retry, queueing). Tail sampling requires all spans of a trace to land on the same gateway instance, so put a `loadbalancing` exporter with `routing_key: traceID` in the agent tier. Getting this wrong produces mysteriously incomplete traces and it is a classic.

Semantic conventions

Semantic conventions are the agreed names for common attributes. They are the reason a Datadog dashboard built for one service works for another, and the reason the Collector's processors can find the fields they need. Use them; do not invent `http_status` when `http.response.status_code` exists.

Domain	Key attributes
Resource (every span)	`service.name`, `service.version`, `service.namespace`, `deployment.environment`, `host.name`, `k8s.pod.name`
HTTP	`http.request.method`, `http.route`, `http.response.status_code`, `url.full`, `server.address`, `server.port`, `network.protocol.version`
Database	`db.system`, `db.namespace`, `db.query.text`, `db.operation.name`, `db.collection.name`
Messaging	`messaging.system`, `messaging.destination.name`, `messaging.operation.type`, `messaging.message.id`, `messaging.batch.message_count`
RPC	`rpc.system`, `rpc.service`, `rpc.method`, `rpc.grpc.status_code`
GenAI	`gen_ai.system`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `gen_ai.response.finish_reasons`
Errors	`error.type`, `exception.type`, `exception.message`, `exception.stacktrace`

✓ **The vendor-neutrality argument, for the customer's architect**

You will be asked "why OpenTelemetry and not just the Datadog agent?" The answer that lands: "Your instrumentation becomes a permanent asset rather than a contract dependency. If you renegotiate with your APM vendor in two years, you change one Collector config file instead of re-instrumenting four hundred services. The instrumentation is in your codebase, under your control, and it outlives any vendor relationship — including ours." Enterprise architects care deeply about lock-in; this is one of the rare occasions where the technically correct answer is also the commercially persuasive one.

SLIs, SLOs and Error Budgets

An SLO is a number that converts an argument into a decision. Without one, "is the system reliable enough?" is a matter of whoever spoke most recently; with one, it is arithmetic. For an FDE this is doubly valuable, because the SLO is the contract boundary between "we delivered what we promised" and "we owe you work" — and because negotiating it forces the customer to say out loud what they actually need, which is frequently different from what they first asked for.

The three terms, precisely

SLI — SERVICE LEVEL INDICATOR

A measurement of one aspect of service quality, expressed as a ratio of good events to valid events. Example: proportion of HTTP requests to `/v1/documents` that return non-5xx within 2 seconds.

SLO — SERVICE LEVEL OBJECTIVE

A target for an SLI over a window. Example: 99.5% over a rolling 28 days. Internal. This is what drives engineering decisions.

SLA — SERVICE LEVEL AGREEMENT

A contractual promise with financial consequences. Always looser than the SLO. If the SLO is 99.5%, the SLA might be 99.0%, so you have headroom to fix things before money is involved.

Choosing a good SLI

A good SLI is one a user would recognise as describing their experience. It must be measurable from a vantage point that includes everything the user depends on, and it must have a clearly defined denominator.

SLI type	Good	Bad	Why
Availability	Non-5xx responses / all valid requests	Pod uptime percentage	A running pod returning 500s is not available
Latency	Requests completing under 2s / all successful requests	Mean response time	Mean hides the tail; a threshold count is a ratio you can budget
Freshness	Documents indexed within 10 min of upload / all uploads	Pipeline ran today	Users care about data being current, not about jobs executing
Correctness	Invoices matched with confidence > 0.9 / all invoices	No exceptions thrown	Silent wrongness is the worst failure mode and needs its own SLI
Throughput	Batches completing within the window / batches scheduled	Messages consumed	Consumption without completion is not success

⚠ **Define the denominator with a lawyer's precision**
"Valid requests" needs a definition before the first incident, not during it. Are 4xx client errors in the denominator? (Usually yes for 429 and 503, no for 400 and 404 — a malformed request is not your outage.) Are health checks? (No.) Are synthetic probes? (No, measure separately.) Are requests during an agreed maintenance window? (No, if the window was announced.) Write it down in the SLO document. Every reliability dispute I have seen with a customer was in fact a dispute about the denominator.

The error budget

TEXT

```
ERROR BUDGET = 1 − SLO, expressed as allowed bad events

  SLO 99.9% over 28 days
  → budget = 0.1% of requests
  → at 500 rps: 500 * 60 * 60 * 24 * 28 = 1,209,600,000 requests
  → budget   = 1,209,600 failed requests
  → or, in time terms: 40 minutes 19 seconds of total unavailability

THE "NINES" TABLE — memorise the middle rows

  SLO       per day        per week      per 28 days     per year
  90%       2h 24m         16h 48m       2d 19h          36d 12h
  99%       14m 24s        1h 41m        6h 43m          3d 15h
  99.5%     7m 12s         50m 24s       3h 22m          1d 19h
  99.9%     1m 26s         10m 05s       40m 19s         8h 46m
  99.95%    43s            5m 02s        20m 10s         4h 23m
  99.99%    8.6s           1m 01s        4m 02s          52m 36s
  99.999%   0.86s          6.0s          24s             5m 15s

WHAT THE BUDGET IS FOR
  - Spending it on releases, experiments, migrations and chaos tests is CORRECT.
  - A budget consistently unspent means the SLO is too loose or you are
    over-investing in reliability and under-investing in features.
  - A budget exhausted means feature work stops and reliability work starts.
    That rule has to be agreed BEFORE it is needed, in writing, or it will be
    overruled the first time it is inconvenient.
```

99.99% is almost never what the customer needs

When a customer says "we need four nines", ask what the business impact of four minutes of downtime per month actually is. Usually: nothing, because the humans using the system are asleep for a third of it, the upstream ERP they depend on is itself 99.5%, and their own network has an unplanned outage every quarter. An SLO cannot exceed that of your weakest dependency. If you consume an API that is 99.9%, you cannot be 99.99% — the arithmetic does not permit it. Walk them through the dependency chain and the number comes down to something achievable within an hour.

Multi-window multi-burn-rate alerting

Naive SLO alerting — "page when the error ratio over 5 minutes exceeds 0.1%" — is terrible. It fires constantly on tiny blips and it tells you nothing about whether you are actually in trouble. Burn rate fixes this: it measures how fast you are consuming the error budget relative to the rate that would exactly exhaust it over the window.

TEXT

```
BURN RATE = observed error ratio / (1 - SLO)

  SLO 99.9% → budget ratio 0.001
  Observed error ratio 0.014  →  burn rate 14x
  At 14x you exhaust a 28-day budget in 28/14 = 2 days.

THE GOOGLE SRE RECOMMENDED TABLE (for a 30-day window)

  Severity   Long window   Short window   Burn rate   Budget consumed   Notify
  ─────────  ────────────  ─────────────  ──────────  ────────────────  ──────────
  CRITICAL   1 hour        5 minutes      14.4x       2%  in 1 hour     Page now
  CRITICAL   6 hours       30 minutes     6x          5%  in 6 hours    Page now
  WARNING    1 day         2 hours        3x          10% in 1 day      Ticket
  WARNING    3 days        6 hours        1x          10% in 3 days     Ticket

WHY TWO WINDOWS PER RULE
  The LONG window gives significance  (is this real, or 3 requests?)
  The SHORT window gives recovery     (stop alerting quickly once it's fixed)
  Both must breach for the alert to fire. The short window is conventionally
  1/12th of the long one.
```

YAML

```yaml
groups:
  - name: docpipeline.slo
    rules:
      # ─── Page: burning 2% of a 30d budget in 1 hour ──────────────
      - alert: DocPipelineErrorBudgetBurnFast
        expr: |
          (
            service:sli_error:ratio_rate1h{service="doc-ingest"}  > (14.4 * 0.001)
            and
            service:sli_error:ratio_rate5m{service="doc-ingest"}  > (14.4 * 0.001)
          )
        for: 2m
        labels:
          severity: critical
          slo: availability
          team: platform
        annotations:
          summary: "doc-ingest is burning its error budget 14x too fast"
          description: >-
            Error ratio is {{ $value | humanizePercentage }} over 1h.
            At this rate the 30-day error budget is exhausted in ~2 days.
          runbook_url: "https://runbooks.acme.corp/docpipeline/error-budget-burn"
          dashboard_url: "https://grafana.acme.corp/d/fde-docpipe-svc?var-service=doc-

      # ─── Page: slower but sustained burn ─────────────────────────
      - alert: DocPipelineErrorBudgetBurnMedium
        expr: |
          (
            service:sli_error:ratio_rate6h{service="doc-ingest"}  > (6 * 0.001)
            and
            service:sli_error:ratio_rate30m{service="doc-ingest"} > (6 * 0.001)
          )
        for: 15m
        labels: { severity: critical, slo: availability }
        annotations:
          summary: "doc-ingest burning error budget 6x too fast over 6h"
          runbook_url: "https://runbooks.acme.corp/docpipeline/error-budget-burn"

      # ─── Ticket: slow burn, deal with it on Monday ──────────────
      - alert: DocPipelineErrorBudgetBurnSlow
        expr: |
          (
            service:sli_error:ratio_rate1d{service="doc-ingest"} > (3 * 0.001)
            and
            service:sli_error:ratio_rate2h{service="doc-ingest"} > (3 * 0.001)
          )
        for: 1h
        labels: { severity: warning, slo: availability }
```

```
    annotations:
      summary: "doc-ingest slow error budget burn — raise a ticket"

  # ─── Latency SLO: 99% of requests under 2 seconds ──────────────
  - alert: DocPipelineLatencySLOBurn
    expr: |
      (
        1 - (
          sum(rate(http_request_duration_seconds_bucket{service="doc-ingest",le="2
          / sum(rate(http_request_duration_seconds_count{service="doc-ingest"}[1h]
        ) > (14.4 * 0.01)
      )
      and
      (
        1 - (
          sum(rate(http_request_duration_seconds_bucket{service="doc-ingest",le="2
          / sum(rate(http_request_duration_seconds_count{service="doc-ingest"}[5m]
        ) > (14.4 * 0.01)
      )
    for: 2m
    labels: { severity: critical, slo: latency }

  # ─── Budget remaining, as a gauge for the dashboard ────────────
  - record: service:error_budget_remaining:ratio30d
    expr: |
      1 - (
        sum(increase(http_requests_total{service="doc-ingest",status=~"5.."}[30d])
        / clamp_min(sum(increase(http_requests_total{service="doc-ingest"}[30d])),
      ) / 0.001
```

Negotiating an SLO with a customer

1. **Start from the user journey, not the service.**
   "When a claims handler uploads a document, they expect to search it within ten minutes." That sentence contains an SLI (freshness), a threshold (10 min) and a population (uploaded documents).

2. **Establish the baseline before promising anything.**
   Run for two to four weeks and measure. Proposing an SLO before you have data is how you end up committed to a number your architecture cannot reach.

3. **Map the dependency chain and compute the ceiling.**
   If you depend on their SharePoint (which their own team measures at 99.5%) and their Entra ID (99.9%) and an LLM endpoint (99.9%), serial dependencies multiply: 0.995 × 0.999 × 0.999 ≈

99.3%. You cannot promise 99.9% end-to-end. Show them this multiplication; it reframes the conversation from "your software" to "our system".

4 **Separate the SLOs you control from those you do not.**
Define one SLI excluding upstream failures ("availability of our processing given a successful fetch") and one including them. Commit tightly to the first, report transparently on the second.

5 **Agree the exclusions explicitly.**
Announced maintenance windows, customer-caused incidents (they revoked the service account, they changed a firewall rule), force majeure at the cloud provider, and load above an agreed ceiling.

6 **Agree the consequence.**
What happens when the budget is exhausted? "Feature work pauses, we do a joint review, we agree remediation." Write it down. An SLO with no consequence is a decoration.

7 **Review it quarterly.**
SLOs are not permanent. Traffic patterns change, dependencies change, business priorities change.

✦ **The sentence that ends most SLO arguments**
"Let's agree a number we can both see on the same dashboard, measured the same way, before we agree what it should be." Shared measurement first, target second. Half the time the customer's assumed baseline is wildly different from the measured one in either direction, and resolving that is more valuable than the negotiation itself.

Alerting That Does Not Cry Wolf

Every alert is an interruption of a human being's life. That framing is not sentimentality — it is the correct economic model, because a team that receives twelve alerts a night will, within a month, have muted all twelve, including the one that mattered. Alert fatigue is not a people problem; it is a design failure, and it is reliably caused by alerting on causes instead of symptoms.

Symptom alerts, not cause alerts

Cause alert (mostly bad)	Symptom alert (good)	Why
CPU > 80%	p99 latency > SLO threshold	High CPU with fine latency is just efficient hardware use
A pod restarted	Availability SLI burning budget	Kubernetes restarting a pod is the system working as designed
Memory > 85%	OOMKills occurring, or latency degraded	JVMs and Go runtimes are supposed to use the memory you gave them
Disk 70% full	Predicted full within 4 hours	70% full and stable for a year is not an incident
Queue has messages	Oldest message older than X, or queue growing for Y minutes	A queue with messages in it is a queue doing its job
An error was logged	Error *rate* above baseline	Distributed systems produce errors continuously and survive

> **The legitimate cause alerts**
>
> There are exactly three categories where alerting on a cause is right: **(1)** conditions that are certain to cause an outage soon and that a symptom alert would catch too late to prevent — disk filling, certificate expiring in 14 days, licence expiring, quota about to be hit; **(2)** conditions that are invisible in symptoms because something is masking them — all replicas of a redundant pair down to one, backup job not run in 48h, replication lag growing; **(3)** security events. Everything else should be a dashboard panel, not a page.

Severity levels with real definitions

Severity	Definition	Response	Examples
SEV1	Complete outage or data loss affecting all users	Page immediately, 24×7, wake people, incident commander	Service returning 500 for everyone; data corruption detected; security breach
SEV2	Major degradation or a large subset of users affected	Page during business hours, notify out of hours	One tenant completely broken; p99 at 10× SLO; SLO budget burning at 14×
SEV3	Minor degradation, workaround exists	Ticket, next business day	Slow burn on budget; a non-critical feature failing; retries elevated
SEV4	Cosmetic or informational	Backlog	A dashboard panel broken; a log message is unclear

The anatomy of an alert that gets acted on

YAML

```yaml
- alert: IngestQueueStalled
  # 1. The expression is a SYMPTOM, with a condition that excludes the benign case
  expr: |
    (ingest_queue_oldest_message_age_seconds > 900)
    and
    (rate(ingest_processed_total[10m]) < 0.1)
  # 2. 'for' eliminates flapping. Set it to ~2x your normal recovery time.
  for: 10m
  labels:
    severity: critical
    service: doc-ingest
    team: platform
    # 3. Routing metadata the customer's Alertmanager/ServiceNow uses
    customer: acme
    environment: prod
    page: "true"
  annotations:
    # 4. Summary: what is wrong, where, how bad. Readable on a phone lock screen.
    summary: "Ingest queue stalled: oldest message {{ $value | humanizeDuration }} old
    # 5. Description: impact in business terms, not technical terms
    description: >-
      The {{ $labels.queue }} queue on {{ $labels.cluster }} has not been
      drained for over 15 minutes and consumers are processing fewer than
      0.1 msg/s. Claims handlers uploading documents will not see them in
      search. Approximately {{ with query "ingest_queue_depth" }}{{ . | first | value
      documents are affected.
    # 6. Runbook: the FIRST THREE THINGS to check, not a wiki homepage
    runbook_url: "https://runbooks.acme.corp/docpipeline/queue-stalled"
    # 7. Dashboard: pre-filtered to this service and a sensible time range
    dashboard_url: "https://grafana.acme.corp/d/fde-docpipe-svc?var-service=doc-ingest
    # 8. Logs: a one-click query
    logs_url: "https://splunk.acme.corp/search?q=index%3Dvendor_docpipeline%20event%3D
```

⚠ **An alert without a runbook is a prank**

Every single alert you create must link to a document that tells a stranger the first three things to check and the first safe remediation. Not "investigate the issue". Literally: "1. Check the consumer pods are running: `kubectl get pods -l app=doc-worker`. 2. Check RabbitMQ is reachable from a worker: `kubectl exec … -- nc -zv rabbit 5672`. 3. If consumers are up and the broker is reachable, check for a poison message: `…`. If none of these, escalate to X." Write the runbook as you write the alert, in the same pull request. Alerts whose runbooks are written "later" never get runbooks.

Runbook template

TEXT

```
# RUNBOOK: IngestQueueStalled

## What this means
The document ingest queue is not draining. Uploaded documents are not
becoming searchable. Users see "processing" indefinitely.

## Business impact
Claims handlers cannot search documents uploaded in the last N minutes.
No data is lost — messages are durable. This is a delay, not a loss.
Escalate to SEV1 only if age exceeds 2 hours or messages are being dropped.

## Diagnose — in this order, stop when you find it
1. Are the consumers running?
   kubectl -n docpipeline get pods -l app=doc-worker
   Expect 3 Running. If CrashLoopBackOff → see "Consumer crash" below.

2. Is the broker healthy and reachable?
   kubectl -n docpipeline exec deploy/doc-worker -- nc -zv rabbit.acme.corp 5672
   Check RabbitMQ management UI: https://rabbit.acme.corp:15672 (creds in vault)

3. Is a single message poisoning the consumer? (same message_id repeating)
   Splunk: index=vendor_docpipeline event=doc.failed | stats count by message_id
   If one id dominates → move it to the DLQ (step 2 under Remediate).

4. Is an upstream dependency down, causing every message to time out?
   Grafana panel "Upstream latency p95". Check probe_success for sharepoint.

5. Is the consumer blocked on a database lock?
   SELECT pid, state, wait_event_type, wait_event, query, now()-query_start AS dur
     FROM pg_stat_activity WHERE state <> 'idle' ORDER BY dur DESC LIMIT 20;

## Remediate
1. Consumer crash: kubectl -n docpipeline rollout restart deploy/doc-worker
   Watch: kubectl logs -f -l app=doc-worker --tail=50
2. Poison message: use the admin endpoint to shovel it to the DLQ
   curl -XPOST $ADMIN/queues/ocr/messages/$ID/dlq -H "Authorization: Bearer $T"
3. Upstream down: nothing to do on our side. Scale consumers to 1 to stop
   hammering the upstream, notify the SharePoint team (see escalation).
   kubectl -n docpipeline scale deploy/doc-worker --replicas=1
4. Genuine backlog: scale out. Safe up to 12 replicas (SharePoint throttles
   above that — see rate limit note in the architecture doc).
   kubectl -n docpipeline scale deploy/doc-worker --replicas=8

## Escalation
L1 → L2 platform (Teams #acme-platform, rota in ServiceNow)
SharePoint issues → ACME Collaboration Services team, queue COLLAB-OPS
Vendor (us) → support@vendor.example, ref contract ACME-2026-DOCP, SLA 4h
```

```
## Verify recovery
- ingest_queue_oldest_message_age_seconds back under 60
- rate(ingest_processed_total[5m]) > 0
- Spot-check: upload a test document via /v1/documents and search for it
```

Pager hygiene

- [ ] Target fewer than two pages per on-call shift. More than that and you must fix the alerts, not the humans.
- [ ] Every page must be **actionable** (there is something to do), **urgent** (it cannot wait until morning) and **real** (not a known flake).
- [ ] Review every alert that fired at the weekly operations meeting. For each: was it actionable? If not, delete it or demote it to a ticket. This meeting is the single most effective alert-fatigue cure.
- [ ] Track the ratio of alerts that resulted in action. Below about 70% and trust is eroding.
- [ ] Use inhibition rules so a single root cause produces one page, not forty.
- [ ] Silences must have an expiry and a reason. An unbounded silence is a deleted alert with extra steps.
- [ ] Test the paging path end to end before go-live — send a real test alert at 11pm and confirm a phone rings. Half of all new on-call setups have a broken integration key that nobody discovers until the first real incident.
- [ ] Alert on the alerting: `up{job="prometheus"} == 0` evaluated by a different Prometheus, or a dead-man's-switch alert that always fires and whose absence pages you.

YAML

```yaml
# The dead man's switch - the alert that proves alerting works.
# It fires permanently; an external watchdog pages if it ever STOPS arriving.
- alert: Watchdog
  expr: vector(1)
  labels: { severity: none }
  annotations:
    summary: "Alerting pipeline is alive. Absence of this alert means it is not."

# Route it to a heartbeat endpoint (PagerDuty heartbeat, Healthchecks.io,
# Dead Man's Snitch, or an Azure Monitor scheduled query on the customer's side).
# Without this, a Prometheus that dies silently takes your entire alerting with it
# and nobody finds out until a customer phones.
```

Debugging Distributed Systems

At some point the dashboards will not tell you the answer and you will have to think. This section is a method, not a tool list. Having a method matters most precisely when you are under pressure in someone else's environment with four people watching your screen share — because that is when ad-hoc poking turns into two hours of flailing.

The method

1. **Establish the blast radius.**
   Everyone, or one tenant? One region, or all? One endpoint, or all? New requests, or also retries of old ones? Narrowing this takes two minutes with a well-built dashboard and eliminates ninety per cent of hypotheses immediately.

2. **Establish the timeline.**
   When exactly did it start? Not "this morning" — the minute. Zoom the graph until you can see the inflection. This single number is the most powerful filter you have.

3. **Ask what changed at that minute.**
   Deploys, config edits, feature flags, certificate rotations, scheduled jobs, upstream releases, DNS changes, firewall changes, licence expiries, month-end batch runs, daylight-saving transitions. The customer's change calendar matters as much as your git log.

4. **Form one hypothesis and state it out loud.**
   "I think the connection pool is exhausted because the SharePoint latency increase is holding connections open." A stated hypothesis is falsifiable; a vague hunch is not.

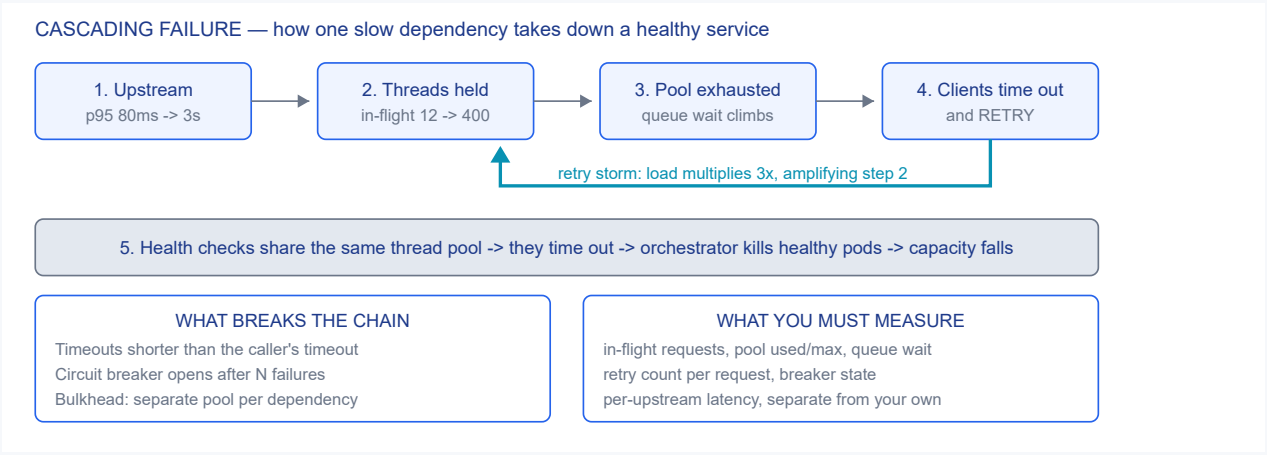
5. **Find the cheapest test that falsifies it.**
   One PromQL query, one log search, one trace. If the test takes more than five minutes to run, find a cheaper test.

6. **Bisect.**
   Across the stack (is it the client, the gateway, the app, the DB?), across time (git bisect over deploys), across population (does it affect only large documents? only one tenant? only PDF?).

7. **Write down what you have ruled out.**
   In the incident channel, as you go. Prevents you and three colleagues from each independently re-testing the same hypothesis, which is what actually happens otherwise.

8. **Stabilise before you understand.**

Roll back, scale out, failover, disable the feature flag. Root cause can wait; the customer's users cannot. Resist the strong engineer's urge to fully understand first.



A cascading failure. Note that nothing in your code was wrong — only its reaction to a slow neighbour.

Timeouts: the most common configuration bug in enterprise deployments

Default timeouts in HTTP clients are either infinite or enormous. `requests` in Python has no default timeout at all — a hung connection blocks the thread forever. The Java `HttpURLConnection` default is infinite. A pod with 200 threads and no timeout can be fully consumed by one unresponsive upstream in under a minute.

TEXT

```
THE TIMEOUT BUDGET — every layer must be shorter than the one above it

  User's browser                                30s
    → Customer's F5 / API gateway               25s
      → Your ingress / load balancer            20s
        → Your service's own request deadline   15s
          → Outbound call to SharePoint          5s  (x2 attempts = 10s worst case)
          → Outbound call to LLM                 8s
          → Database query                       3s  (statement_timeout)
          → Redis                              200ms

  IF A LOWER LAYER TIMES OUT AFTER AN UPPER ONE, work continues on a request
  nobody is waiting for. The user sees a 504, retries, and you now have two
  copies of the work running. Repeat and you have a self-inflicted DDoS.

  CHECK THIS EXPLICITLY. Ask the customer for their gateway timeout. It is
  frequently 30s or 60s and frequently nobody knows without looking.
```

PYTHON

```python
# Every outbound call, every time. No exceptions.
import httpx, random, time

client = httpx.Client(
    timeout=httpx.Timeout(
        connect=2.0,    # TCP + TLS handshake
        read=5.0,       # time between bytes
        write=5.0,
        pool=1.0,       # time waiting for a free connection — FAIL FAST here
    ),
    limits=httpx.Limits(max_connections=50, max_keepalive_connections=20,
                        keepalive_expiry=30.0),
    transport=httpx.HTTPTransport(retries=0),   # we handle retries ourselves
)

def call_with_backoff(url: str, max_attempts: int = 3, deadline_s: float = 15.0):
    """Exponential backoff with FULL JITTER and a hard overall deadline."""
    started = time.monotonic()
    last_exc = None
    for attempt in range(1, max_attempts + 1):
        remaining = deadline_s - (time.monotonic() - started)
        if remaining ≤ 0:
            raise DeadlineExceeded(f"budget exhausted after {attempt-1} attempts")
        try:
            r = client.get(url, timeout=httpx.Timeout(min(5.0, remaining)))
            if r.status_code == 429:
                # ALWAYS honour Retry-After. Enterprise APIs mean it.
                wait = float(r.headers.get("Retry-After", 2 ** attempt))
                UPSTREAM_ERRORS.labels("sharepoint", "429").inc()
            elif 500 ≤ r.status_code < 600:
                wait = random.uniform(0, min(2 ** attempt, 20))   # full jitter
                UPSTREAM_ERRORS.labels("sharepoint", str(r.status_code)).inc()
            else:
                return r                                          # 2xx/4xx: done
        except (httpx.ConnectTimeout, httpx.ReadTimeout, httpx.ConnectError) as e:
            last_exc = e
            wait = random.uniform(0, min(2 ** attempt, 20))
            UPSTREAM_ERRORS.labels("sharepoint", type(e).__name__).inc()

        if attempt == max_attempts:
            break
        log.warning("upstream.retry", upstream="sharepoint", attempt=attempt,
                    wait_s=round(wait, 2), remaining_budget_s=round(remaining, 2))
        time.sleep(min(wait, max(0.0, remaining - 0.1)))
    raise UpstreamUnavailable("sharepoint") from last_exc
```

Retries multiply load exactly when you can least afford it

Three retries at every layer of a four-layer stack is $3^4 = 81$ requests for one user action. When an upstream is struggling, retries are the mechanism by which it goes from degraded to dead. Three rules: **(1)** retry at exactly one layer, usually the outermost one that knows the operation is idempotent; **(2)** always use full jitter — `sleep(random(0, backoff))`, not `sleep(backoff)`, or every client retries in synchronised waves; **(3)** use a retry budget — stop retrying globally when retries exceed ~10% of total requests, because at that point retrying is making things worse for everyone.

Circuit breakers and bulkheads

PYTHON

```python
import threading, time
from enum import Enum

class State(Enum):
    CLOSED = "closed"        # normal: requests flow
    OPEN = "open"            # tripped: fail fast, do not call upstream
    HALF_OPEN = "half_open"  # probing: allow a trickle to test recovery

class CircuitBreaker:
    def __init__(self, name, failure_threshold=5, success_threshold=2,
                 timeout_s=30, window_s=60):
        self.name = name
        self.failure_threshold = failure_threshold
        self.success_threshold = success_threshold
        self.timeout_s = timeout_s
        self.window_s = window_s
        self._lock = threading.Lock()
        self._state = State.CLOSED
        self._failures: list[float] = []
        self._successes = 0
        self._opened_at = 0.0

    def _set_state(self, new: State):
        if new is not self._state:
            log.warning("circuit.state_change", breaker=self.name,
                        old=self._state.value, new=new.value)
            BREAKER_STATE.labels(self.name).set(
                {State.CLOSED: 0, State.HALF_OPEN: 1, State.OPEN: 2}[new])
        self._state = new

    def call(self, fn, *args, **kwargs):
        with self._lock:
            now = time.monotonic()
            if self._state is State.OPEN:
                if now - self._opened_at ≥ self.timeout_s:
                    self._set_state(State.HALF_OPEN)
                    self._successes = 0
                else:
                    BREAKER_REJECTED.labels(self.name).inc()
                    raise CircuitOpen(f"{self.name} open, retry in "
                                      f"{self.timeout_s - (now - self._opened_at):.0f}
        try:
            result = fn(*args, **kwargs)
        except Exception:
            with self._lock:
                now = time.monotonic()
                self._failures = [t for t in self._failures if now - t < self.window_s
                self._failures.append(now)
```

```
                if self._state is State.HALF_OPEN or \
                   len(self._failures) ≥ self.failure_threshold:
                    self._set_state(State.OPEN)
                    self._opened_at = now
            raise
        else:
            with self._lock:
                if self._state is State.HALF_OPEN:
                    self._successes += 1
                    if self._successes ≥ self.success_threshold:
                        self._set_state(State.CLOSED)
                        self._failures.clear()
                else:
                    self._failures.clear()
            return result

# Bulkhead: a SEPARATE connection/thread pool per dependency, so a slow
# SharePoint cannot consume the capacity needed to serve cached requests.
sharepoint = CircuitBreaker("sharepoint", failure_threshold=5, timeout_s=30)
llm        = CircuitBreaker("llm", failure_threshold=3, timeout_s=60)

def fetch(doc_id):
    try:
        return sharepoint.call(_fetch_impl, doc_id)
    except CircuitOpen:
        # GRACEFUL DEGRADATION: serve something rather than nothing.
        # This is what the customer remembers about your software.
        cached = cache.get(doc_id)
        if cached:
            log.info("served.degraded", doc_id=doc_id, reason="circuit_open")
            return cached
        raise
```

✦ **Expose breaker state as a metric and put it on the dashboard**
`circuit_breaker_state{name="sharepoint"}` as a gauge (0/1/2) is one of the most informative single panels you can build. When it flips to 2, you instantly know the problem is upstream, not you — and so does the customer's L1 operator, without needing to understand your code.

Backpressure and queue depth

A system without backpressure converts overload into collapse. When demand exceeds capacity you have three options and you must pick one deliberately: queue it (bounded, with a maximum age), shed it

(reject with 429 and a `Retry-After`), or degrade it (serve a cheaper result). Doing nothing means unbounded queue growth, memory exhaustion, and an OOM kill that loses everything in flight.

TEXT

```
QUEUE METRICS THAT MATTER (depth alone is almost useless)

  queue_depth                    how many are waiting          (lagging indicator)
  queue_oldest_message_age       how long the worst one waited  ⟵ THE SLI
  queue_arrival_rate             msgs/sec in
  queue_service_rate             msgs/sec out
  queue_wait_time_histogram      distribution of waiting time

  LITTLE'S LAW:   L = λ × W      (depth = arrival rate × wait time)
  Rearranged:     W = L / λ

  Worked example: 4,000 messages queued, consumers processing 12/sec
                  → 4000 / 12 = 333 seconds = 5.5 minutes to drain
                  → if arrival rate is 15/sec, it NEVER drains. Scale or shed.

  ALERT ON:  oldest_message_age > SLO    (symptom)
             arrival_rate > service_rate sustained 10m   (leading indicator)
  NOT ON:    depth > 1000   (meaningless without knowing the service rate)
```

PYTHON

```python
# Load shedding at the edge: reject early, cheaply, with a useful response.
from fastapi import FastAPI, Request, Response
import asyncio

MAX_IN_FLIGHT = 200
_sema = asyncio.Semaphore(MAX_IN_FLIGHT)

@app.middleware("http")
async def shed_load(request: Request, call_next):
    if request.url.path in ("/healthz", "/readyz", "/metrics"):
        return await call_next(request)          # NEVER shed health checks
    if _sema.locked():
        SHED_TOTAL.labels(request.url.path).inc()
        return Response(
            status_code=503,
            headers={"Retry-After": "5",
                     "X-Shed-Reason": "max-in-flight"},
            content='{"error":"overloaded","retry_after":5}',
            media_type="application/json")
    async with _sema:
        IN_FLIGHT.inc()
        try:
            return await call_next(request)
        finally:
            IN_FLIGHT.dec()
```

> ⚠ **Health checks must not share the saturated resource**
>
> If your `/readyz` endpoint queries the database and the database is slow, Kubernetes will mark every replica unready and remove them all from the load balancer — turning a degradation into a total outage. Liveness should check only "is this process responsive" (no dependencies at all). Readiness may check dependencies, but should have its own short timeout and should ideally not flap. Startup probes exist for slow-booting applications; use them rather than making liveness lenient.

Profiling and Performance

Traces tell you which service is slow. Profiles tell you which line of code. In a customer environment where you cannot attach a debugger to production and cannot reproduce the workload locally, continuous profiling is frequently the only way to find out why a process is burning 90% of a CPU doing apparently nothing.

Types of profile

Profile	Measures	Finds	Typical overhead
CPU (sampling)	Stacks sampled at ~100 Hz	Hot loops, regex backtracking, JSON parsing, crypto, compression	1–3%
Wall clock / off-CPU	Time blocked, not just time running	Lock contention, I/O waits, thread starvation	2–5%
Heap / allocation	Bytes and objects allocated	Memory leaks, allocation churn driving GC	1–5%
Goroutine / thread	Count and stack of each	Leaked goroutines, blocked threads, deadlocks	<1%
Lock / mutex	Contention time per lock	Serialisation under concurrency	2–10%

Reading a flame graph

TEXT

```
HOW TO READ IT — this takes thirty seconds to learn and never stops being useful

  x-axis  = NOT time. It is alphabetically sorted stacks, merged. Width = sample count
  y-axis  = stack depth. Caller below, callee above.
  Width   = total time in that function AND its children (inclusive).
  Plateau = a wide frame with nothing wide above it = time spent IN that function
            (exclusive). THIS IS WHERE THE WORK IS. Look for flat tops.

  WHAT TO LOOK FOR
    A wide plateau near the top          → the hot function. Optimise or cache it.
    A wide tower of many thin frames     → deep call stack, often framework overhead
    json.loads / json.dumps wide         → serialising too much, or too often
    re.match / regex wide                → catastrophic backtracking; check the patte
    gc / GC frames wide                  → allocation pressure, not the GC's fault
    ssl / crypto wide                    → connection churn; enable keep-alive
    time.sleep / select / epoll wide     → you are looking at a WALL-CLOCK profile;
                                           that is waiting, not burning

  DIFFERENTIAL FLAME GRAPHS are the real power move: render v2.7.0 vs v2.7.1 with
  frames coloured by delta. Red = got slower. You find a regression in seconds.
```

BASH

```
# --- Linux, any language, no code changes, production-safe ──────────────
# Install once; works on Go, Rust, C/C++, Java (with -XX:+PreserveFramePointer),
# and Node (--perf-basic-prof)
sudo perf record -F 99 -a -g -p $(pgrep -f doc-ingest) -- sleep 30
sudo perf script > out.perf
./FlameGraph/stackcollapse-perf.pl out.perf | ./FlameGraph/flamegraph.pl > cpu.svg

# --- Python: py-spy, the single most useful tool in this list ────────────
pip install py-spy
py-spy top --pid 1234                       # live top-like view, zero restart
py-spy record -o flame.svg --pid 1234 -d 60 --subprocesses
py-spy dump --pid 1234                      # stack of EVERY thread, right now
                                            # → this is how you find a deadlock
# In a container (needs SYS_PTRACE or a shared pid namespace):
kubectl exec -it doc-ingest-7f9c8-4xq2l -- py-spy dump --pid 1

# --- Java ────────────────────────────────────────────────────────────────
jcmd $PID Thread.print                      # full thread dump
jcmd $PID GC.heap_info
jcmd $PID JFR.start duration=60s filename=/tmp/rec.jfr settings=profile
jcmd $PID JFR.stop name=1                   # then open in JDK Mission Control
jmap -dump:live,format=b,file=/tmp/heap.hprof $PID    # analyse in Eclipse MAT
async-profiler/profiler.sh -d 60 -e cpu -f flame.html $PID

# --- Node ────────────────────────────────────────────────────────────────
node --cpu-prof --cpu-prof-dir=/tmp app.js          # .cpuprofile for Chrome devtool
kill -SIGUSR1 $PID                                  # open the inspector live
npx clinic doctor -- node app.js                    # guided diagnosis
# Heap snapshot in production without a restart:
node --inspect=0.0.0.0:9229 app.js   # then chrome://inspect → Memory → snapshot

# --- Go ──────────────────────────────────────────────────────────────────
import _ "net/http/pprof"                            // exposes /debug/pprof
go tool pprof -http=:8080 http://localhost:6060/debug/pprof/profile?seconds=30
go tool pprof http://localhost:6060/debug/pprof/heap
go tool pprof http://localhost:6060/debug/pprof/goroutine     # leak hunting
curl -s localhost:6060/debug/pprof/goroutine?debug=2 | head -100
```

py-spy dump is the deadlock detector

When a Python service stops responding but does not crash and CPU is near zero, `py-spy dump --pid 1` prints the current stack of every thread without attaching a debugger, without restarting, and without the GIL cooperating. Nine times out of ten you will see every worker thread parked on the same `acquire()` or the same socket `recv()` and the diagnosis is immediate. Build it into your image, or ensure you can `kubectl debug` with an ephemeral container that has it.

Continuous profiling

Continuous profiling collects low-frequency samples from every process all the time and stores them for weeks. The overhead is 1–2% and the payoff is that when a customer says "it was slow last Tuesday afternoon", you can simply look. Pyroscope (now part of Grafana), Parca, Polar Signals and Datadog/Dynatrace all do this; the OpenTelemetry profiling signal is standardising it.

YAML

```yaml
# Grafana Alloy / Pyroscope eBPF profiler as a DaemonSet — no app changes at all.
# This is the version to propose when you cannot modify the customer's services.
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: pyroscope-ebpf, namespace: observability }
spec:
  template:
    spec:
      hostPID: true                    # required to see other processes' stacks
      containers:
        - name: alloy
          image: grafana/alloy:latest
          securityContext:
            privileged: true           # eBPF; expect a security review on this
            capabilities: { add: ["SYS_ADMIN", "SYS_PTRACE", "SYS_RESOURCE"] }
          env:
            - { name: PYROSCOPE_SERVER, value: "http://pyroscope:4040" }
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }
```

> ⚠ **Privileged eBPF will not pass every security review**
>
> In regulated customers, a privileged DaemonSet with `hostPID` is a hard no. Have a fallback ready: in-process profilers (pyroscope-io SDK, pprof endpoints, JFR) that require no elevated privileges, enabled behind a flag, sampling at 10–20 Hz. Knowing both options and which one to propose in which room is part of the job.

Garbage collection and memory

TEXT

```
SYMPTOMS AND CAUSES

  p99 latency spikes, p50 flat, sawtooth memory
    → GC pauses. Look at gc_pause_seconds / jvm_gc_pause_seconds_sum.
      Fix: reduce allocation rate, not GC tuning. Allocation is the cause.

  Memory climbs monotonically, never falls, eventually OOMKilled
    → Leak. In Python: unbounded cache, circular refs with __del__, a growing
      module-level list, a logger handler accumulating records.
      In Java: ThreadLocal not cleared, ClassLoader leak, listener not removed.
      In Node: closure capturing a large object, event listener not removed,
      a Map keyed by request id that is never cleaned.

  Memory high but stable at the limit, no OOM
    → Probably fine. The JVM/Go runtime expands to fill what you gave it.
      Alert on OOMKills and on GC time, not on RSS.

  Container OOMKilled but process heap looks small
    → Off-heap: native allocations, thread stacks (1MB each x 500 threads),
      memory-mapped files, JVM Metaspace, glibc arena fragmentation.
      Set MALLOC_ARENA_MAX=2 for JVM/Python in containers; it is a known
      multi-hundred-megabyte win.

KEY NUMBERS TO WATCH
  container_memory_working_set_bytes      ← this is what the OOM killer uses
  container_spec_memory_limit_bytes
  rate(container_memory_failcnt[5m])
  jvm_gc_pause_seconds{quantile="0.99"}   target: < 100ms for an interactive API
  go_gc_duration_seconds
  python: gc.get_count(), tracemalloc snapshots diffed over time
```

PYTHON

```python
# Finding a Python leak in production with tracemalloc, safely
import tracemalloc, gc, linecache

tracemalloc.start(25)                       # keep 25 frames
_baseline = tracemalloc.take_snapshot()

def leak_report(top_n: int = 15) -> str:
    """Call from an admin endpoint. Compares against the baseline."""
    gc.collect()
    snapshot = tracemalloc.take_snapshot().filter_traces((
        tracemalloc.Filter(False, tracemalloc.__file__),
        tracemalloc.Filter(False, linecache.__file__),
    ))
    stats = snapshot.compare_to(_baseline, "lineno")
    lines = []
    for s in stats[:top_n]:
        lines.append(f"{s.size_diff/1024/1024:+8.2f} MB  {s.count_diff:+8d} objs  "
                     f"{s.traceback.format()[-1].strip()}")
    return "\n".join(lines)

# Typical output from a real incident:
#   +412.80 MB    +1840221 objs  File "app/cache.py", line 42: self._store[key] = valu
#   The "cache" had no eviction policy. Three lines to fix; four hours to find
#   without tracemalloc.
```

Database performance: the usual suspect

In the large majority of "the application is slow" investigations in enterprise deployments, the database is involved. Learn to read an execution plan; it is a thirty-minute skill that pays back on the first incident.

SQL

```sql
-- PostgreSQL: the queries to run first, in this order

-- 1. What is running RIGHT NOW and how long has it been running?
SELECT pid, now() - query_start AS duration, state, wait_event_type, wait_event,
       left(query, 120) AS query
FROM pg_stat_activity
WHERE state ≠ 'idle' AND query NOT ILIKE '%pg_stat_activity%'
ORDER BY duration DESC
LIMIT 20;

-- 2. Is anything blocked on a lock?
SELECT blocked.pid AS blocked_pid, blocked.query AS blocked_query,
       blocking.pid AS blocking_pid, blocking.query AS blocking_query,
       now() - blocked.query_start AS blocked_for
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;

-- 3. Aggregate worst offenders (requires pg_stat_statements)
SELECT round(total_exec_time::numeric, 0) AS total_ms,
       calls,
       round(mean_exec_time::numeric, 2) AS mean_ms,
       round((100 * total_exec_time / sum(total_exec_time) OVER ())::numeric, 1) AS pc
       rows / GREATEST(calls, 1) AS rows_per_call,
       left(query, 100) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
-- Look for: huge `calls` with tiny mean_ms → that is your N+1.
--           high mean_ms with few calls   → that is your missing index.

-- 4. Unused and missing indexes
SELECT schemaname, relname, indexrelname, idx_scan, pg_size_pretty(pg_relation_size(in
FROM pg_stat_user_indexes WHERE idx_scan < 50 ORDER BY pg_relation_size(indexrelid) DE

SELECT relname, seq_scan, seq_tup_read, idx_scan,
       seq_tup_read / GREATEST(seq_scan, 1) AS avg_rows_per_seq_scan
FROM pg_stat_user_tables
WHERE seq_scan > 1000 ORDER BY seq_tup_read DESC LIMIT 20;
```

SQL

```
-- Reading EXPLAIN (ANALYZE, BUFFERS) — the only form worth running
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT d.id, d.title, t.name
FROM documents d
JOIN tenants t ON t.id = d.tenant_id
WHERE d.created_at > now() - interval '7 days'
  AND d.status = 'indexed'
ORDER BY d.created_at DESC
LIMIT 50;

/*
 Limit  (cost=184221.44..184221.56 rows=50 width=68)
        (actual time=3124.881..3124.901 rows=50 loops=1)
   Buffers: shared hit=1204 read=98441
   ->  Sort  (cost=184221.44..184942.09 rows=288260 width=68)
             (actual time=3124.878..3124.886 rows=50 loops=1)
         Sort Key: d.created_at DESC
         Sort Method: top-N heapsort  Memory: 36kB
         ->  Hash Join  (cost=28.70..174634.11 rows=288260 width=68)
                        (actual time=0.412..2981.223 rows=291104 loops=1)
               Hash Cond: (d.tenant_id = t.id)
               ->  Seq Scan on documents d                    <-- PROBLEM 1
                     (cost=0.00..169821.00 rows=288260 width=52)
                     (actual time=0.019..2704.881 rows=291104 loops=1)
                     Filter: ((created_at > (now() - '7 days')) AND (status = 'indexed
                     Rows Removed by Filter: 4208896            <-- read 4.5M, kept 2
                     Buffers: shared read=98441                 <-- 98k blocks = 770
               ->  Hash  (cost=18.20..18.20 rows=840 width=24)
 Planning Time: 0.284 ms
 Execution Time: 3125.004 ms

 WHAT TO READ, IN ORDER
 1. "Execution Time" at the bottom — the real number.
 2. Compare `rows=` (estimate) with `actual rows=` — a 10x+ divergence means
    the planner is working from stale statistics. Run ANALYZE.
 3. "Rows Removed by Filter" — 4.2 million rows read and discarded. That is a
    missing index, every time.
 4. "Seq Scan" on a large table with a selective filter — missing index.
 5. "Buffers: read=" — blocks fetched from disk, 8KB each. 98441 x 8KB = 770MB
    of I/O for 50 rows.
 6. "loops=N" on a nested node with N in the thousands — that is an N+1.
 7. "Sort Method: external merge Disk: 420MB" — work_mem too small.

 THE FIX
   CREATE INDEX CONCURRENTLY idx_documents_status_created
     ON documents (status, created_at DESC)
     WHERE status = 'indexed';          -- partial index: smaller and faster
```

```
 AFTER
   Limit (actual time=0.067..0.291 rows=50 loops=1)
     →  Index Scan Backward using idx_documents_status_created on documents
   Execution Time: 0.412 ms              -- 7,600x faster
*/
```

CREATE INDEX CONCURRENTLY, always, in production

A plain `CREATE INDEX` takes an `ACCESS EXCLUSIVE` lock and blocks all writes to the table for the duration — which on a 200 GB table is tens of minutes and is an outage. `CONCURRENTLY` takes longer and cannot run inside a transaction, but does not block. The same applies to `ALTER TABLE ... ADD COLUMN ... DEFAULT` on older PostgreSQL, and to anything involving `ALTER TYPE`. Before you run DDL on a customer's production database, check the lock level required and say out loud what it will block.

The N+1 query, the most common performance bug in the world

PYTHON

```python
# THE BUG — 1 query for the list, then 1 query per item. 201 round trips.
docs = session.query(Document).limit(200).all()
for d in docs:
    print(d.tenant.name)        # lazy load: SELECT * FROM tenants WHERE id = ?

# In a trace this is unmistakable: 200 identical thin spans in a staircase.
# At 2ms RTT inside a datacentre that is 400ms. Across a customer's WAN to a
# database in another region at 40ms RTT, it is EIGHT SECONDS.

# THE FIX — eager load, 2 queries total
from sqlalchemy.orm import joinedload, selectinload
docs = (session.query(Document)
        .options(selectinload(Document.tenant))   # 1 extra query with IN (...)
        .limit(200).all())

# DETECT IT AUTOMATICALLY — fail the test suite rather than production
import sqlalchemy.event as event

class QueryCounter:
    def __init__(self): self.count = 0
    def __call__(self, conn, cur, stmt, params, ctx, many): self.count += 1

def test_document_list_is_not_n_plus_one(session):
    counter = QueryCounter()
    event.listen(session.bind, "before_cursor_execute", counter)
    list_documents(session, limit=200)
    event.remove(session.bind, "before_cursor_execute", counter)
    assert counter.count ≤ 3, f"N+1 detected: {counter.count} queries for 200 rows"
```

LLM and AI Observability

AI systems break the assumptions of conventional observability in three specific ways: they are non-deterministic (the same input produces different output), they cost real money per request in a way that is directly proportional to usage, and they fail silently — producing a confident, well-formed, completely wrong answer that no HTTP status code will ever flag. If you are deploying RAG or agentic systems into customer environments, these are the signals that matter.

What to measure that is specific to LLMs

Signal	Metric	Why it matters on site
Cost	`llm_tokens_total{direction,model,tenant_tier}`	The customer is paying per token. An unbounded retrieval context turns a $400/month system into a $40,000 one in a week.
Latency breakdown	`llm_stage_duration_seconds{stage}`	Retrieval vs rerank vs generation. "The AI is slow" is usually "your vector DB is slow" or "we are stuffing 40k tokens of context".
Time to first token	`llm_ttft_seconds`	Perceived latency for streaming UIs. A 400ms TTFT with 8s total feels fast; a 6s TTFT feels broken.
Rate limiting	`llm_rate_limited_total`, headers `x-ratelimit-remaining-*`	Enterprise Azure OpenAI deployments have fixed TPM/RPM quotas. You will hit them.
Retrieval quality	Top-k scores, hit rate, chunks returned, `retrieval_empty_total`	Empty or low-score retrieval is the #1 cause of hallucination in RAG.
Output validity	`llm_output_invalid_total{reason}`	JSON that does not parse, schema violations, refusals, truncation at max_tokens.

Groundedness	Sampled LLM-as-judge or citation-coverage score	The only proxy for "is it making things up" you can compute continuously.
Feedback	Thumbs up/down rate, edit rate, abandonment	The only ground truth you will ever get. Instrument the UI for it on day one.
Drift	Embedding distribution shift, query-type mix, refusal rate over time	The model is updated under you; the corpus changes; user behaviour changes.

PYTHON

```python
# Full instrumentation of a RAG chain with OpenTelemetry GenAI conventions.
from opentelemetry import trace, metrics
from opentelemetry.trace import SpanKind
import hashlib, time, json

tracer = trace.get_tracer("rag-service", "1.4.0")
meter  = metrics.get_meter("rag-service", "1.4.0")

tokens   = meter.create_counter("gen_ai.client.token.usage", unit="{token}")
cost_usd = meter.create_counter("llm.cost.usd", unit="USD")
ttft     = meter.create_histogram("llm.time_to_first_token", unit="s",
             explicit_bucket_boundaries_advisory=[0.1,0.25,0.5,1,2,4,8,16])
retr_hist= meter.create_histogram("rag.retrieval.score",
             explicit_bucket_boundaries_advisory=[0,0.2,0.4,0.5,0.6,0.7,0.8,0.9,1.0])

PRICE = {  # USD per 1M tokens — keep this in config, it changes
    "gpt-4o":        {"in": 2.50, "out": 10.00},
    "gpt-4o-mini":   {"in": 0.15, "out":  0.60},
    "claude-sonnet": {"in": 3.00, "out": 15.00},
}

def answer(question: str, tenant: str) -> dict:
    with tracer.start_as_current_span("rag.answer") as root:
        root.set_attribute("tenant.id", tenant)
        root.set_attribute("rag.question.length", len(question))
        # NEVER the question text itself unless the customer has signed off on it.
        root.set_attribute("rag.question.hash",
                           hashlib.sha256(question.encode()).hexdigest()[:16])

        # ─── 1. Embed ──────────────────────────────────────────────
        with tracer.start_as_current_span("rag.embed", kind=SpanKind.CLIENT) as s:
            s.set_attribute("gen_ai.system", "azure.openai")
            s.set_attribute("gen_ai.request.model", "text-embedding-3-large")
            t0 = time.perf_counter()
            qvec = embed(question)
            s.set_attribute("gen_ai.usage.input_tokens", est_tokens(question))
            s.set_attribute("duration_ms", (time.perf_counter()-t0)*1000)

        # ─── 2. Retrieve ───────────────────────────────────────────
        with tracer.start_as_current_span("rag.retrieve", kind=SpanKind.CLIENT) as s:
            s.set_attribute("db.system", "pgvector")
            s.set_attribute("rag.top_k", 20)
            hits = vector_search(qvec, k=20, tenant=tenant)
            scores = [h.score for h in hits]
            s.set_attribute("rag.hits.count", len(hits))
            s.set_attribute("rag.hits.max_score", max(scores) if scores else 0.0)
            s.set_attribute("rag.hits.min_score", min(scores) if scores else 0.0)
            s.set_attribute("rag.doc_ids", json.dumps([h.doc_id for h in hits][:10]))
```

```
        for sc in scores:
            retr_hist.record(sc, {"tenant.tier": tier(tenant)})
        # The single most valuable RAG alert
        if not hits or max(scores) < 0.45:
            s.add_event("rag.retrieval.weak", {"max_score": max(scores, default=0)
            RETRIEVAL_WEAK.labels(tenant_tier=tier(tenant)).inc()

    # ─── 3. Rerank ──────────────────────────────────────────────────
    with tracer.start_as_current_span("rag.rerank") as s:
        top = rerank(question, hits)[:5]
        s.set_attribute("rag.reranked.count", len(top))
        s.set_attribute("rag.rerank.dropped",
                        len([h for h in hits if h.doc_id not in
                             {t.doc_id for t in top}]))

    # ─── 4. Generate ────────────────────────────────────────────────
    model = "gpt-4o"
    with tracer.start_as_current_span("rag.generate", kind=SpanKind.CLIENT) as s:
        s.set_attribute("gen_ai.system", "azure.openai")
        s.set_attribute("gen_ai.operation.name", "chat")
        s.set_attribute("gen_ai.request.model", model)
        s.set_attribute("gen_ai.request.temperature", 0.1)
        s.set_attribute("gen_ai.request.max_tokens", 1024)
        s.set_attribute("rag.context.chunks", len(top))
        s.set_attribute("rag.context.tokens", sum(c.tokens for c in top))
        # Prompt VERSION, not prompt TEXT — this is how you correlate a
        # quality regression with a prompt change weeks later.
        s.set_attribute("llm.prompt.version", PROMPT_VERSION)
        s.set_attribute("llm.prompt.template_hash", PROMPT_HASH)

        t0 = time.perf_counter()
        first_token_at = None
        chunks = []
        for delta in stream_chat(model, build_prompt(question, top)):
            if first_token_at is None:
                first_token_at = time.perf_counter() - t0
                s.set_attribute("gen_ai.time_to_first_token", first_token_at)
                ttft.record(first_token_at, {"gen_ai.request.model": model})
            chunks.append(delta)
        out = "".join(chunks)
        usage = last_usage()

        s.set_attribute("gen_ai.usage.input_tokens",  usage.prompt_tokens)
        s.set_attribute("gen_ai.usage.output_tokens", usage.completion_tokens)
        s.set_attribute("gen_ai.response.finish_reasons", [usage.finish_reason])
        s.set_attribute("gen_ai.response.id", usage.id)

        attrs = {"gen_ai.request.model": model, "tenant.tier": tier(tenant)}
        tokens.add(usage.prompt_tokens,     {**attrs, "gen_ai.token.type": "input"
```

```
        tokens.add(usage.completion_tokens, {**attrs, "gen_ai.token.type": "output
        cost = (usage.prompt_tokens     * PRICE[model]["in"]
              + usage.completion_tokens * PRICE[model]["out"]) / 1_000_000
        cost_usd.add(cost, attrs)
        s.set_attribute("llm.cost.usd", round(cost, 6))

        if usage.finish_reason == "length":
            s.add_event("llm.output.truncated")
            TRUNCATED.labels(model).inc()

    # ─── 5. Validate ──────────────────────────────────────────────
    with tracer.start_as_current_span("rag.validate") as s:
        citations = extract_citations(out)
        covered = len([c for c in citations if c in {t.doc_id for t in top}])
        s.set_attribute("rag.citations.count", len(citations))
        s.set_attribute("rag.citations.grounded", covered)
        s.set_attribute("rag.citations.coverage",
                        covered / len(citations) if citations else 0.0)
        if citations and covered < len(citations):
            s.add_event("rag.citation.hallucinated",
                        {"claimed": len(citations), "grounded": covered})
            HALLUCINATED_CITATION.inc()

    root.set_attribute("llm.cost.usd", round(cost, 6))
    return {"answer": out, "sources": [t.doc_id for t in top],
            "trace_id": format(root.get_span_context().trace_id, "032x")}
```

✕ **Logging prompts and completions: get this in writing**

Prompts in a customer deployment contain the customer's data — claim details, patient notes, contract terms, salaries. Logging them wholesale into an observability backend creates an unapproved copy of regulated data in a system with different access controls and different retention. Default to: **hash the input, log the length, log the prompt template version, log the token counts, log nothing of the content.** If you need content for debugging, make it opt-in per tenant, time-boxed, with explicit written approval, into a separate store with short retention and restricted access — and put the fact that it is enabled on the dashboard so nobody forgets.

Cost control, which the customer will ask about in week two

TEXT

```
PROMQL FOR THE COST CONVERSATION

  Spend rate, dollars per hour, right now
    sum(rate(llm_cost_usd_total[5m])) * 3600

  Projected monthly spend at the current rate
    sum(rate(llm_cost_usd_total[1h])) * 3600 * 24 * 30

  Which tenant is expensive
    topk(5, sum by (tenant_tier) (increase(llm_cost_usd_total[24h])))

  Cost per successful answer — the number that actually matters
    sum(increase(llm_cost_usd_total[1h]))
      / sum(increase(rag_answers_total{outcome="success"}[1h]))

  Input/output token ratio — if input dwarfs output you are stuffing context
    sum(rate(gen_ai_client_token_usage_total{gen_ai_token_type="input"}[1h]))
      / sum(rate(gen_ai_client_token_usage_total{gen_ai_token_type="output"}[1h]))
    Healthy RAG: 10:1 to 40:1.  Above 100:1 → your retrieval is too greedy.

  Rate-limit headroom against an Azure OpenAI TPM quota
    (sum(rate(gen_ai_client_token_usage_total[1m])) * 60) / 240000   # 240k TPM PTU

ALERTS WORTH HAVING
  - Projected monthly spend > budget * 1.2           (warning, ticket)
  - Hourly spend > 3x the 7-day hourly median        (critical — runaway loop)
  - Cache hit rate drops below 30%                    (someone changed the key)
  - Mean input tokens per request up 50% week-on-week  (context creep)
```

⚠ **The agentic loop is a financial denial-of-service**

An agent that can call tools can loop. A loop that calls an LLM costs money per iteration. I have seen a single malformed user query produce 1,100 LLM calls in nine minutes. Always set: a hard maximum iteration count, a per-request token budget enforced in code, a per-tenant hourly spend cap, and a span attribute `agent.iteration` so the loop is visible in the trace. Alert on `max(agent_iterations) > threshold`. This is not hypothetical and the customer will not forgive the invoice.

Quality signals and drift

ONLINE EVALUATION

Sample 1–5% of production responses and score them automatically: groundedness (does every claim appear in the retrieved context?), citation coverage, answer relevance, refusal rate, schema validity. Run the judge on a cheap model. Store the scores as metrics so you can alert on a regression.

IMPLICIT FEEDBACK

Did the user copy the answer? Edit it? Ask the same question again, rephrased? Abandon the session? Click a citation? These are free, unbiased and far more plentiful than thumbs-up buttons. Instrument them in the UI before go-live.

DRIFT DETECTION

Track the distribution of question embeddings, the mix of question types, the retrieval score distribution, and the refusal rate week over week. A sudden shift means either the users changed, the corpus changed, or the model changed under you — which happens silently with hosted model endpoints.

GOLDEN SET REGRESSION

Keep 100–300 question/expected-answer pairs agreed with the customer. Run them on every deploy, on every prompt change, and weekly against production. This is the only artefact that lets you say "the quality has not regressed" with evidence rather than vibes.

✓ **Pin the model version and alert on it changing**

`gpt-4o` is a moving target; `gpt-4o-2024-11-20` is not. Always pin to a dated deployment, record the exact model in a span attribute and a `build_info`-style metric, and alert when the observed model string changes. "Quality dropped last Thursday and nothing in our code changed" is almost always a silently updated model endpoint, and you will spend three days on it if you have not recorded the version.

Incident Response and Incident Command

An incident is a social process with a technical substrate. The failure mode of unstructured incident response is not that nobody knows what to do — it is that six capable people all do the same thing simultaneously while nobody talks to the customer. Incident command is a small amount of structure that converts a crowd into a team. As an FDE you will often be the most senior technical person in the room on your product and the least senior person in the organisation, which makes knowing this structure disproportionately valuable.

Roles

Role	Does	Does NOT
Incident Commander (IC)	Owns the incident. Assigns work, makes decisions, keeps the timeline, decides severity, declares resolution.	Debug. The moment the IC starts typing into a terminal, the incident loses its coordinator.
Operations / Ops Lead	The person actually making changes. Reports findings and proposes actions to the IC.	Make unilateral production changes, or talk to stakeholders.
Communications Lead	Status page, stakeholder updates, customer emails, exec briefing. Updates on a fixed cadence.	Debug, or speculate publicly about cause.
Scribe	Timestamped log of every observation, hypothesis, action and decision in the incident channel.	Editorialise. Record, do not judge.
Subject matter experts	Pulled in as needed, released when done.	Stay on the call indefinitely "in case".

> **In a small deployment, one person wears several hats — but name them**
> Even alone, say out loud: "I am IC and ops. I will post an update every 15 minutes." It changes your behaviour. When the incident grows, hand off the IC role explicitly and announce it: "Priya is now IC." Ambiguous command is how two people roll back to different versions.

The first fifteen minutes

1. **0–2 min — Acknowledge and declare.**
   Ack the page. Open an incident channel (`#inc-2026-03-14-docpipeline`). State: "Declaring SEV2. I am IC. Symptom: 14% 5xx on /v1/documents since 09:07." Declaring early is free; un-declaring is easy. Hesitating costs you the first ten minutes.

2. **2–5 min — Scope it.**
   Who is affected, how many, which operations, which regions, which tenants. One dashboard, thirty seconds per question. Write the answers in the channel.

3. **5–7 min — First customer communication.**
   Before you know the cause. "We are aware of elevated errors affecting document upload starting 09:07. We are investigating. Next update at 09:30." Silence is interpreted as incompetence; an update with no answer is interpreted as control.

4. **7–10 min — What changed.**
   Check deploys, config, flags, certs, upstream status pages, the customer's change calendar. More than half of incidents end here.

5. **10–15 min — Mitigate, do not fix.**
   Roll back, failover, scale, disable the flag, shed load, fail over to a cached path. The goal is to stop the bleeding. Root cause analysis happens after the users are OK.

6. **Throughout — Keep the timeline.**
   Every action with a timestamp, in the channel, as it happens. Reconstructing it afterwards from memory is unreliable and you will need it for the postmortem.

TEXT

```
INCIDENT CHANNEL — what good looks like

09:11  @priya  Declaring SEV2. I am IC. 14% of POST /v1/documents returning 500
               since 09:07. All tenants. Dashboard: <link>
09:12  @priya  @tom you are ops. @sara you are comms. I am scribe for now.
09:13  @tom    Error code is E_SP_TIMEOUT. SharePoint p95 went 180ms → 4.1s at 09:06.
09:14  @sara   Posted status: "investigating elevated errors on document upload,
               next update 09:30"
09:15  @priya  Hypothesis 1: SharePoint-side issue. Hypothesis 2: our connection
               pool. @tom check probe_success for sharepoint from blackbox.
09:17  @tom    blackbox probe to sharepoint.acme.corp also at 4s. It is NOT us.
               Ruling out hypothesis 2.
09:18  @priya  Confirmed upstream. Mitigation: open the circuit breaker manually
               and serve from cache. Degrades freshness, keeps the service up.
               @tom proceed.
09:21  @tom    Breaker forced open. 5xx rate dropping: 14% → 0.4%.
09:23  @priya  @sara please update: mitigated, serving cached results, working
               with ACME Collaboration Services on the SharePoint side.
09:24  @priya  @sara also raise a ticket with ACME COLLAB-OPS, ref our incident id.
09:30  @sara   Status updated. ACME ticket COLLAB-884213 raised, they have
               acknowledged a SharePoint farm issue on their side.
10:02  @tom    SharePoint p95 back to 210ms. Closing breaker.
10:15  @priya  Error rate 0.02% for 13 minutes. Declaring resolved. Postmortem
               scheduled Thursday 14:00, I will draft by Wednesday EOD.
```

Communicate on a clock, not on events

Promise an update time and hit it even when you have nothing new. "09:30 update: still investigating, no new information, next update 10:00" maintains trust far better than an eventual brilliant explanation at 11:15. Stakeholders escalate when they feel uninformed, and an escalation during an incident costs you your best engineer for twenty minutes.

Status updates: the template

TEXT

```
[INVESTIGATING]  14 Mar 09:12 UTC
We are investigating elevated error rates affecting document upload.
Impact: approximately 14% of uploads are failing. Documents already
uploaded are unaffected and no data has been lost.
Workaround: retry the upload; most second attempts succeed.
Next update: 09:30 UTC.

[IDENTIFIED]  14 Mar 09:24 UTC
We have identified the cause as degraded performance in an upstream
document storage system. We have enabled a fallback that restores
upload functionality. Search results may be up to 30 minutes stale
until the upstream system recovers.
Next update: 10:00 UTC.

[MONITORING]  14 Mar 10:05 UTC
The upstream system has recovered. Error rates have returned to normal
and the fallback has been disabled. We are monitoring.
Next update: 10:45 UTC or on resolution.

[RESOLVED]  14 Mar 10:15 UTC
This incident is resolved. Total impact: 09:07–09:21 UTC, approximately
2,400 failed upload attempts, all of which could be retried successfully.
No data was lost. A postmortem will be published within 5 working days.

RULES
  - Impact in the user's language, never in yours. "Uploads failing", not
    "E_SP_TIMEOUT on the ingest path".
  - State what is NOT affected. It is as valuable as what is.
  - Give a workaround if one exists, in the first update.
  - Never name a vendor or blame a team in a customer-facing update.
  - Never speculate about cause before you have evidence.
  - Commit to a next-update time in every single update.
```

Blameless Postmortems

The purpose of a postmortem is to change the system so that this class of incident is less likely or less damaging. It is not to identify who made a mistake. This is not kindness — it is epistemology: in an environment where naming a culprit is the outcome, people stop volunteering information, and the information you lose is exactly the information you needed. A blameless culture is the cheapest reliability investment available.

The template

TEXT

```
# POSTMORTEM: Document upload failures due to upstream SharePoint degradation
Incident ID: INC-2026-0314-01    Severity: SEV2    Status: Resolved
Authors: P. Shah (IC), T. Okafor (Ops)        Review date: 2026-03-19

## SUMMARY  (3 sentences, readable by an executive)
Between 09:07 and 09:21 UTC on 14 March, 14% of document upload requests
failed with HTTP 500. The cause was a 20x latency increase in ACME's
SharePoint farm, which exhausted our connection pool. No data was lost
and all failed uploads were retryable.

## IMPACT
- Duration: 14 minutes of user-visible impact (09:07–09:21)
- Users affected: all tenants; approx 180 claims handlers
- Requests failed: ~2,400 of 17,100 (14.0%)
- Error budget consumed: 31% of the 28-day budget in 14 minutes
- Data loss: none. Financial impact: none. SLA breach: no (SLA 99.0%, month at 99.4%)

## TIMELINE  (all times UTC, from the incident channel — do not reconstruct from memor
09:06  ACME SharePoint farm p95 latency rises 180ms → 4.1s (their change window,
       see CONTRIBUTING FACTOR 3)
09:07  Our connection pool (max 50) saturates. Queue wait > 5s. First 500s.
09:09  Alert DocPipelineErrorBudgetBurnFast fires. Paged on-call.
09:11  Incident declared SEV2.
09:13  E_SP_TIMEOUT identified as the dominant error code.
09:17  Blackbox probe confirms the latency is upstream, not in our code.
09:21  Circuit breaker manually forced open; serving from cache. Errors → 0.4%.
09:30  ACME Collaboration Services confirm a farm-level issue on their side.
10:02  SharePoint recovers. Breaker closed.
10:15  Resolved.

## ROOT CAUSE
Not a single cause. The upstream latency increase was the trigger; the
reason it became a user-visible outage was that our connection pool had
no timeout on pool acquisition, so requests queued indefinitely rather
than failing fast, and the circuit breaker threshold (5 failures in 60s)
was never reached because requests were hanging rather than failing.

## CONTRIBUTING FACTORS  (there are always several)
1. Pool acquisition had no timeout (httpx Limits without pool timeout).
2. Circuit breaker counted exceptions, not slow responses. A dependency
   that is slow but not failing never trips it.
3. ACME's SharePoint patching window was not on any calendar we had access to.
4. The fallback-to-cache path existed but was not automatic; it required a
   manual flag flip, costing 4 minutes.
5. Our runbook did not mention the cache fallback. The responder knew about
   it from memory.

## WHAT WENT WELL
```

```
- The burn-rate alert fired within 2 minutes of user impact.
- The blackbox probe immediately ruled out our own network, saving ~15 minutes
  of investigation and avoiding a wrong-direction rollback.
- Customer communication went out at 09:14, before the cause was known.

## WHAT WAS LUCKY  (name this explicitly — luck is not a control)
- It happened at 09:07 on a weekday with a senior engineer already online.
  At 03:00 on a Sunday the mitigation would have taken 40 minutes, not 14.

## ACTION ITEMS
| # | Action                                                    | Owner | Due    | Prio
|---|-----------------------------------------------------------|-------|--------|-----
| 1 | Add pool acquisition timeout (1s) to all HTTP clients     | T.O.  | 21 Mar | P0
| 2 | Circuit breaker trips on latency p95 as well as errors    | T.O.  | 28 Mar | P0
| 3 | Automatic cache fallback when breaker opens (no manual)   | P.S.  | 04 Apr | P1
| 4 | Add cache-fallback step to the queue-stalled runbook      | P.S.  | 18 Mar | P0
| 5 | Request read access to ACME change calendar; add as       | P.S.  | 28 Mar | P1
|   | Grafana annotations                                       |       |        |
| 6 | Add alert on upstream p95 latency, not just error rate    | T.O.  | 28 Mar | P1
| 7 | Game day: simulate 5s SharePoint latency in staging       | T.O.  | 18 Apr | P2

## LESSONS
Slow is worse than down. A dependency that fails fast is survivable; a
dependency that hangs consumes every resource you have. Every timeout in
the stack must be explicit, and every circuit breaker must consider latency.
```

> ⚠ **Action items that actually get done**
>
> Three rules, learned the hard way. **(1)** A named individual, never a team — "the platform team" owns nothing. **(2)** A date, and the date is tracked in the same backlog as feature work, with the same visibility. **(3)** No more than seven items, ranked, with at most two P0s. A postmortem with twenty-three action items produces zero completed action items; one with four produces four. Review completion at the next postmortem — "we said we would do X after the last incident; we did not, and here we are" is the most persuasive argument for prioritisation ever invented.

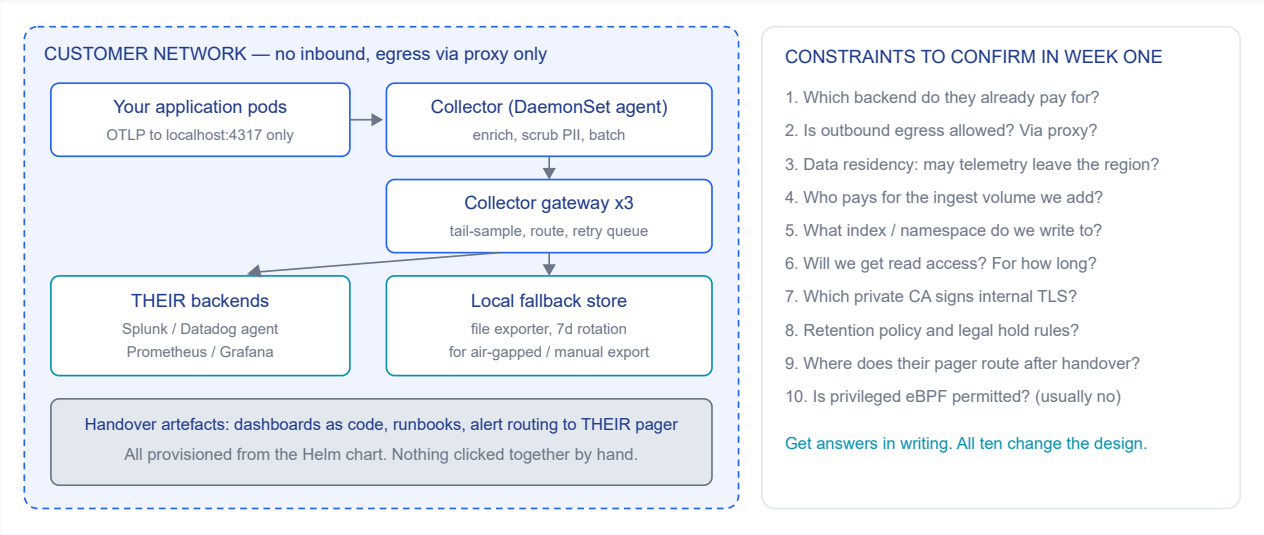
Running the review

- [ ] Within five working days, while memory is fresh and before the next incident overwrites it.
- [ ] The draft is circulated 24 hours before the meeting. People who have read it ask better questions.
- [ ] Start by reading the timeline out loud. It is remarkable how often someone says "wait, that is not what I thought happened".

- [ ] Ask "what information did you have at that moment?" rather than "why did you do that?". The second question is an accusation wearing a question mark.
- [ ] Include the customer where the relationship allows it. A joint postmortem with the customer's team after a shared incident builds more trust than any amount of successful uptime.
- [ ] Publish it. An internal wiki page nobody can find is not published. Link it from the runbook for the alert that fired.
- [ ] Counterfactuals are banned: "if only he had checked the dashboard" is not analysis, it is hindsight. "The dashboard did not make the queue age visible" is analysis.

Observability Inside a Customer's Environment

Everything above assumes you can choose your tools. In a customer's environment you usually cannot. This section is the practical compromise layer: how to get the signals you need out of a system you do not own, into a backend you did not choose, under constraints you did not set, and leave it working after you are gone.



A telemetry topology that survives a customer's network policy and your eventual departure.

Shipping into their backend

Their backend	How you get in	Watch out for
Splunk	HEC endpoint + token, or their Universal Forwarder reading stdout	Index must be pre-created and sized; they bill by GB/day ingested; sourcetype must be agreed or search-time field extraction will not work
Datadog	OTLP into their Agent (port 4317) or `datadog` exporter with an API key	Custom metrics are billed per series — this is where cardinality costs real money; agent version may be old and lack OTLP ingest
Elastic / OpenSearch	`elasticsearch` exporter, or Filebeat/Fluent Bit reading stdout	Index templates and ILM policy must exist first; mapping explosions from dynamic JSON fields will break their cluster — set `dynamic: strict` or use a flattened field
Azure Monitor / App Insights	`azuremonitor` exporter, or the Application Insights SDK	Sampling defaults are aggressive and surprising; workspace may be in a different region (data residency); connection string in Key Vault
Dynatrace	OneAgent (auto-injects) or OTLP endpoint + API token	OneAgent is invasive and may conflict with your own instrumentation — check for double-instrumented spans
Prometheus + Loki + Tempo	`prometheusremotewrite`, `loki`, `otlp` exporters	Usually the easiest. Confirm tenant ID header (`X-Scope-OrgID`) for multi-tenant Mimir/Loki
ServiceNow (for alerts)	Alertmanager webhook to their Event Management API	Field mapping must be agreed; incident auto-creation rules are owned by their ITSM team and take weeks to change
Nothing at all	You ship a minimal Prometheus + Grafana + Loki in the Helm chart	You now own an observability stack's lifecycle. Set retention low, set resource limits, document the upgrade path, and get them to adopt it

Ask who pays before you send the first byte

Observability platforms are billed on ingest volume. If you add 40 GB/day to a customer's Splunk without telling anyone, somebody gets a surprise on the next invoice and your integration becomes a cost-reduction target. Estimate your volume before deployment (log lines/day × average bytes, plus metric series count, plus sampled trace volume), present the number, get agreement, and then implement sampling and filtering to stay under it. Put a metric on it: `otelcol_exporter_sent_log_records_total` and alert if you exceed the agreed budget.

No-egress and air-gapped environments

1. **Assume no outbound by default.**
   Design so that every component works with zero internet access: pinned images in their registry, no CDN fonts in dashboards, no external schema fetches, no licence phone-home, no vendor SaaS dependency in the critical path.

2. **Deploy the full stack locally.**
   Prometheus (15-day retention), Loki or a file-based log store, Tempo or Jaeger with in-memory/badger storage, Grafana with provisioned dashboards. Size it for their volume and put hard resource limits on everything.

3. **Build a support bundle command.**
   One command that produces a single encrypted archive: last N hours of logs (redacted), a metrics snapshot, the config (secrets stripped), version info, and a summary report. The customer runs it, their security team inspects it, and they send it to you through whatever channel is approved.

4. **Make the bundle reviewable.**
   Plain text or JSON, with a manifest listing every file and why it is there. A security team that cannot inspect what is leaving will not approve it leaving.

5. **Redact at generation, not at transmission.**
   The bundle itself must contain no secrets or PII, because once it is written to disk you have lost control of it.

BASH

```
#!/usr/bin/env bash
# support-bundle.sh — what we ship for air-gapped customers.
# Produces ONE file the customer's security team can inspect before release.
set -euo pipefail

NS="${NAMESPACE:-docpipeline}"
HOURS="${HOURS:-6}"
OUT="support-bundle-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$OUT"/{logs,metrics,config,k8s}

echo "==> versions"
{ echo "bundle_version: 3"; echo "generated_at: $(date -u +%FT%TZ)";
  echo "namespace: $NS"; echo "hours: $HOURS";
  kubectl -n "$NS" get deploy -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.sp
} > "$OUT/MANIFEST.txt"

echo "==> logs (redacted at write time)"
for pod in $(kubectl -n "$NS" get pods -o name); do
  name="${pod#pod/}"
  kubectl -n "$NS" logs "$name" --all-containers --since="${HOURS}h" --timestamps 2>/d
    | sed -E 's/[A-Za-z0-9._%+-]+@[A-Za-z0-9·-]+\.[A-Za-z]{2,}/[EMAIL]/g;
              s/("?(password|token|secret|api_?key|authorization)"?\s*[:=]\s*)"?[^",}
              s/\b[0-9]{13,19}\b/[CARD]/g;
              s/eyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}/[JWT]/g'
    > "$OUT/logs/${name}.log"
  kubectl -n "$NS" logs "$name" --all-containers --previous --since="${HOURS}h" \
    > "$OUT/logs/${name}.previous.log" 2>/dev/null || true
done

echo "==> metrics snapshot"
for q in 'up' \
         'sum by (service,status) (increase(http_requests_total[6h]))' \
         'histogram_quantile(0.99, sum by (le,route) (rate(http_request_duration_secon
         'sum by (upstream,error_code) (increase(upstream_errors_total[6h]))' \
         'ingest_queue_depth' 'ingest_queue_oldest_message_age_seconds' \
         'circuit_breaker_state' \
         'container_memory_working_set_bytes{namespace="'"$NS"'"}' \
         'increase(kube_pod_container_status_restarts_total{namespace="'"$NS"'"}[24h])
do
  safe=$(echo "$q" | tr -cd '[:alnum:]_' | cut -c1-60)
  curl -sG "http://prometheus.${NS}:9090/api/v1/query" --data-urlencode "query=$q" \
    > "$OUT/metrics/${safe}.json" || true
done

echo "==> kubernetes state"
kubectl -n "$NS" get events --sort-by=.lastTimestamp > "$OUT/k8s/events.txt"
kubectl -n "$NS" describe pods                       > "$OUT/k8s/describe-pods.txt"
kubectl -n "$NS" get all -o wide                     > "$OUT/k8s/get-all.txt"
```

```
kubectl -n "$NS" top pods                                > "$OUT/k8s/top-pods.txt" 2>/dev

echo "==> config (secrets excluded by name AND by value)"
kubectl -n "$NS" get configmap -o yaml                   > "$OUT/config/configmaps.yaml"
kubectl -n "$NS" get secret -o name                      > "$OUT/config/secret-names-only

echo "==> integrity + archive"
( cd "$OUT" && find . -type f -exec sha256sum {} \; > SHA256SUMS )
tar czf "${OUT}.tgz" "$OUT" && rm -rf "$OUT"
echo "Bundle: ${OUT}.tgz"
echo "Contents are plain text. Review before sending. No secrets, no PII, no document
```

Data residency, classification and the security review

- [ ] Telemetry is data. If their data cannot leave the EU, neither can their trace attributes.
- [ ] Classify your telemetry explicitly in a one-page document: which fields, what classification, where stored, how long, who can read it. Security teams approve documents, not intentions.
- [ ] Default to hashing identifiers rather than carrying them. `sha256(user_id + per-deployment-salt)` still supports "is this one user or many?" without carrying an identifier.
- [ ] Know the retention: GDPR right-to-erasure applies to logs. If a user exercises it, can the customer find and delete their records? If your logs contain a user identifier, the honest answer must be yes.
- [ ] Name the sub-processors. If telemetry reaches a vendor, that vendor is a sub-processor and must be listed in the DPA. This is a legal blocker that surfaces late and delays go-live by weeks if you have not raised it.
- [ ] Expect to justify every outbound connection, every privileged container, every hostPath mount and every capability. Have a minimal-privilege variant of your deployment ready.

Instrumenting for the engineer who is not you

ERROR MESSAGES ARE A UI

"Connection refused" is useless. "Cannot reach SharePoint at sharepoint.acme.corp:443 — connection refused after 2000ms. Check: (1) firewall rule from namespace docpipeline, (2) the SP_HOST config value, (3) runbook E_SP_CONN." The person reading it has never seen your code.

A STARTUP SELF-TEST

On boot, verify every dependency and log a structured result for each: DNS resolves, TCP connects, TLS validates, credentials authenticate, required permission present. Fail fast and loudly on required ones. This turns a two-day "it does not work" into a thirty-second diagnosis.

A DIAGNOSTICS ENDPOINT

`GET /admin/diagnostics` returning config (redacted), dependency check results, versions, feature flags, circuit breaker states and queue depths — as JSON. Support asks them to paste the output; you get everything in one message.

VERSION EVERYTHING, EVERYWHERE

Version in logs, in metrics (`build_info`), in span resource attributes, in the diagnostics endpoint, in a response header. "Which version are you running?" should never require a Kubernetes command.

PYTHON

```python
# Startup self-test — the highest ROI 60 lines you will write for a deployment
import socket, ssl, time
from dataclasses import dataclass, asdict

@dataclass
class CheckResult:
    name: str; required: bool; ok: bool
    detail: str; duration_ms: float; remediation: str = ""

def check(name, required, fn, remediation=""):
    t0 = time.perf_counter()
    try:
        detail = fn() or "ok"
        ok = True
    except Exception as e:
        detail, ok = f"{type(e).__name__}: {e}", False
    return CheckResult(name, required, ok, detail,
                       round((time.perf_counter()-t0)*1000, 1),
                       remediation if not ok else "")

def startup_checks() -> list[CheckResult]:
    results = [
        check("dns.sharepoint", True,
              lambda: socket.gethostbyname(cfg.sp_host),
              "DNS cannot resolve the SharePoint host. Check the pod's DNS config "
              "and whether an internal zone is required (runbook E_DNS)."),
        check("tcp.sharepoint", True,
              lambda: socket.create_connection((cfg.sp_host, 443), timeout=5) and None
              "TCP to sharepoint:443 refused or timed out. Likely a firewall rule. "
              "Required: egress from namespace docpipeline to sharepoint.acme.corp:443
        check("tls.sharepoint", True, _verify_tls_chain,
              "TLS verification failed. If the customer uses an internal CA, mount "
              "it and set SSL_CERT_FILE / REQUESTS_CA_BUNDLE (runbook E_TLS_CA)."),
        check("auth.sharepoint", True, _sp_whoami,
              "Authentication failed. Check the service principal secret has not "
              "expired and that Sites.Read.All admin consent is granted."),
        check("perm.sharepoint.read", True, lambda: _sp_list(cfg.sp_library),
              "Authenticated but cannot list the library. The app registration "
              "needs Sites.Selected granted on this specific site."),
        check("db.connect", True, lambda: db.execute("SELECT 1") and None, ""),
        check("db.migrations", True, _assert_schema_current,
              "Database schema is behind the application. Run the migration job."),
        check("broker.connect", True, lambda: broker.ping(), ""),
        check("llm.endpoint", True, _llm_ping,
              "LLM endpoint unreachable. Check AZURE_OPENAI_ENDPOINT and whether "
              "the proxy requires NO_PROXY for the internal domain."),
        check("otel.collector", False, lambda: socket.create_connection(
                  (cfg.otel_host, 4317), timeout=2) and None,
```

```
            "Telemetry will be buffered then dropped. Non-fatal but we will be blind
    check("clock.skew", False, _ntp_offset_under_2s,
            "Clock skew > 2s will break OAuth token validation and make traces "
            "render out of order."),
]
for r in results:
    (log.error if (not r.ok and r.required) else
     log.warning if not r.ok else log.info)("startup.check", **asdict(r))
failed_required = [r for r in results if r.required and not r.ok]
if failed_required:
    log.critical("startup.aborted",
                 failed=[r.name for r in failed_required],
                 message="Refusing to start. See remediation in the checks above."
    raise SystemExit(1)       # Fail fast. A half-working service is worse.
return results
```

✓ **Handover is a deliverable with an acceptance test**

The handover is not complete when you have shown someone the dashboard. It is complete when a member of the customer's L1 team, with you silent in the room, can: open the service dashboard and state whether it is healthy; find the logs for a failed request given a request ID; follow one runbook end to end; and acknowledge a test alert on their own pager. Run that exercise as a formal session, with a checklist, two weeks before you leave. Everything that fails becomes a work item while you are still there to fix it.

Interview Questions

What is the difference between monitoring and observability, and why does it matter more when you deploy into a customer's environment?

Monitoring is checking predefined conditions — you decide in advance what to measure and what threshold matters, then you watch for breaches. It handles the failure modes you anticipated. Observability is the property of being able to answer new questions about a system's behaviour from its existing outputs, without shipping new code. The practical test is: when a novel problem occurs, can you investigate it with the telemetry you already have? In a customer environment this distinction becomes critical because the feedback loop is broken — you cannot add a log line and redeploy in twenty minutes. A change may require a change-board approval, a maintenance window and weeks of elapsed time. So the cost of under-instrumenting is enormously higher than it is for a SaaS team. The practical consequence is that you instrument richly and with high-cardinality context up front, because the marginal cost of an extra span attribute at build time is effectively zero and the marginal cost of adding one after go-live is a fortnight of calendar time and a credibility hit.

A customer reports that your service is slow. Walk me through how you would investigate.

First I would establish scope and timeline, because those two facts eliminate most hypotheses immediately. Scope: is it all users or one tenant, all endpoints or one, all regions or one? Timeline: zoom the latency graph until I can see the exact minute it changed. Then I would ask what changed at that minute — our deploys, config, feature flags, and critically the customer's own change calendar, because in enterprise environments their patching, certificate rotations and network changes cause as many incidents as our code does. Next I would decompose the latency: I instrument every outbound dependency with its own duration histogram, so I can immediately see whether our processing time is flat while an upstream has degraded. That distinction matters enormously, because "your software is slow" and "your SharePoint went from 180ms to 4 seconds" lead to completely different conversations and I want evidence, not opinion. If the time is genuinely in our code, I go to traces — find a p99 trace and a p50 trace for the same operation and diff them structurally, looking for a wide span with no wide children, for gaps indicating queue or lock wait, or for repeated spans indicating an N+1. If traces point at a single function, I go to a profile. Throughout, I would state each hypothesis explicitly and find the cheapest test that falsifies it, and I would record in the incident channel what I have ruled out so that other people do not re-test the same things.

Explain cardinality and why it matters. Give an example of a mistake you would avoid.

Cardinality is the number of distinct time series, which is the product of all label value combinations for a metric. Each series consumes memory in the scraper, disk in the time-series database, and money with a commercial vendor — roughly 3–4 KB of resident memory per active series in Prometheus, so a million series is several gigabytes before you run a single query. The classic mistake is adding a user ID, session ID, trace ID or full URL path as a metric label. If you label an HTTP latency histogram with user ID and you have 180,000 users, you multiply every bucket by 180,000 and create millions of series, which will OOM the customer's Prometheus or produce a five-figure surprise on their Datadog bill. The rule I apply is: before shipping any label, ask what the maximum number of distinct values is, forever. If the answer is unbounded or grows with users or requests, it belongs on a span attribute or a log field, not a metric label. If I need to slice by user, I use exemplars to link from a metric spike to an actual trace, or I bucket the dimension — `tenant_tier` with three values instead of `tenant_id` with five hundred. I also set defensive limits at scrape time: `sample_limit`, `label_limit`, and `metric_relabel_configs` that drop known-bad labels, so that a careless change in the application cannot take down the customer's monitoring.

What is an error budget and how would you use one in a conversation with a customer?

An error budget is the inverse of an SLO: if the objective is 99.9% availability over 28 days, the budget is 0.1% of requests, which is about forty minutes of total unavailability or, at 500 requests per second, about 1.2 million failed requests. The point is that it converts reliability from a value judgement into arithmetic. It also makes explicit that some unreliability is acceptable and intentional — budget is what you spend on releases, migrations and experiments. In a customer conversation I use it in three ways. First, to right-size expectations: when someone asks for four nines, I walk through the dependency chain and show that if we depend on their SharePoint at 99.5% and their identity provider at 99.9%, serial dependencies multiply and the ceiling is around 99.3%, so four nines is

arithmetically impossible regardless of how good our code is. That reframes the discussion from "your software" to "our system" and usually produces a realistic number within an hour. Second, to agree a consequence up front: when the budget is exhausted, feature work pauses and reliability work starts. An SLO with no consequence is decoration. Third, to drive alerting: I implement multi-window multi-burn-rate alerts rather than threshold alerts, so we page when we are burning budget fourteen times too fast over a one-hour and a five-minute window simultaneously, which is both significant and fast to recover.

Why do you prefer histograms over summaries, and how do you choose buckets?

Because you cannot aggregate percentiles. A summary computes quantiles on the client, so if instance A reports a p99 of 100ms and instance B reports 900ms, there is no valid operation that combines those into a fleet p99 — the true value is unknowable from those two numbers. Histograms ship raw bucket counts, which can be summed across instances and then inverted into a quantile with `histogram_quantile`, so they work correctly in any system with more than one replica, which is all real systems. The price is that you must choose bucket boundaries in advance, and the defaults in most libraries are tuned for sub-second HTTP APIs. I choose buckets by placing four to six below the SLO threshold and three or four above it, spanning the realistic range of the operation. For a document pipeline with a two-second target and a tail out to two minutes I would use 0.1, 0.25, 0.5, 1, 2, 3, 5, 10, 30, 60, 120. If I used the defaults, which top out at 10 seconds, everything above that would land in the infinity bucket and the computed p99 would be meaningless. I also keep the bucket count modest, because the series count is the number of label combinations times buckets plus two, and that multiplies quickly.

How does distributed tracing work, and where does it typically break?

A trace represents one request as a tree of spans. Each span has an ID, a parent ID, a name, a start time, a duration and attributes; they all share a trace ID. Distribution works by context propagation: the caller serialises its span context into a transport header — the W3C `traceparent` header, which encodes version, trace ID, parent span ID and sampling flags — and the callee extracts it and makes its spans children of that context. The things that break it are predictable. Message queues break it because headers are not carried unless you explicitly inject them into message properties and extract them in the consumer. Thread pools and background executors break it because context is stored in thread-local or async-local storage and does not survive a `submit()`, so you must capture the context on the calling thread and attach it on the worker. Scheduled jobs and callbacks break it for the same reason. API gateways sometimes strip unrecognised headers. Third-party SDKs that build their own HTTP client bypass your instrumentation entirely. And in enterprise environments you frequently meet services using B3, Jaeger or X-Ray formats rather than W3C, so I configure multiple propagators. The symptom of broken propagation is a trace that stops abruptly at a service boundary, or a flood of single-span traces, and I test it explicitly as part of deployment verification rather than assuming it works.

Why is OpenTelemetry particularly important for a Forward Deployed Engineer?

Because it decouples instrumentation from destination, and in this role the destination is not your choice. One customer runs Splunk, the next runs Datadog, the third runs Elastic, the fourth is air-gapped with a Prometheus and nothing else. If I instrument with a vendor SDK, I either rewrite the instrumentation per customer or I force the customer to buy my vendor, and neither is acceptable. With OpenTelemetry the application emits OTLP to a local Collector, and the Collector is where the vendor-specific decision lives — one config file, swapped per deployment. The Collector earns its place for a second reason too: it is where I do the things I cannot do in the application, like enriching with Kubernetes metadata, scrubbing PII before it reaches a backend I do not control, dropping health-check noise that is often forty per cent of span volume, deriving RED metrics from spans for legacy services I cannot modify, and doing tail-based sampling so I keep every error and every slow trace but only a few per cent of the boring ones. There is also a commercial argument that lands with enterprise architects: the instrumentation becomes a permanent asset in their codebase rather than a contract dependency, so they can change APM vendors without re-instrumenting hundreds of services. That is often the argument that gets the design approved.

Describe a cascading failure and how you would design against it.

A cascading failure is when a localised problem propagates through a system by consuming shared resources. The canonical sequence: an upstream dependency slows from eighty milliseconds to three seconds; requests to it hold threads and connections for much longer; the connection pool or thread pool saturates; new requests queue; clients hit their timeouts and retry, multiplying load on a system that is already saturated; health checks that share the same thread pool start timing out, so the orchestrator kills pods that were actually fine, removing capacity at exactly the wrong moment. Note that nothing in the code was wrong — only its reaction to a slow neighbour. The defences are layered. Explicit timeouts at every level, arranged as a budget where each inner layer's timeout is strictly shorter than its caller's, so work never continues on a request nobody is waiting for. Pool acquisition timeouts so you fail fast rather than queueing indefinitely — this is the one people forget. Circuit breakers that trip on latency as well as on errors, because a dependency that is slow but not failing never trips an error-counting breaker. Bulkheads, meaning a separate connection pool per dependency so a slow one cannot starve the others. Retries with full jitter, at exactly one layer, with a global retry budget that stops retrying when retries exceed about ten per cent of traffic. Load shedding at the edge with 429s and a `Retry-After`. And health checks that do not depend on the saturated resource — liveness should check only that the process is responsive, or Kubernetes will turn your degradation into a total outage.

What do you instrument differently for an LLM or RAG application?

Three things change. First, cost becomes a first-class operational signal, because you pay per token and usage scales with adoption. I emit token counters split by input and output, by model and by tenant tier, convert them to dollars with a price table in config, and alert on projected monthly spend and on hourly spend exceeding three times the weekly median — the latter catches runaway agent loops, which are a genuine financial denial-of-service. I also enforce hard iteration caps and per-request token budgets in code, not just in monitoring. Second, latency must be decomposed by stage: embedding, retrieval, reranking and generation, plus time to first token separately from total

duration, because a streaming UI with a four-hundred-millisecond first token feels fast even at eight seconds total. Without that breakdown, "the AI is slow" is unactionable. Third, and most importantly, these systems fail silently — they return a confident, well-formed, wrong answer with a 200 status code. So I instrument quality proxies: retrieval scores and a counter for weak or empty retrieval, which is the leading cause of hallucination; citation coverage, meaning what fraction of cited documents were actually in the retrieved context; output schema validity and truncation; and sampled LLM-as-judge groundedness scores. I also pin the model to a dated version and alert when the observed model string changes, because hosted endpoints get updated silently and "quality dropped and nothing in our code changed" is otherwise a three-day investigation. On privacy, I default to hashing the question, recording its length and the prompt template version, and logging no content at all, because prompts contain the customer's regulated data and logging them creates an unapproved copy in a system with different access controls.

A customer will not give you access to their monitoring system. How do you operate?

This is common and it is a design constraint rather than a blocker. First, I make the telemetry legible to someone else: structured JSON with stable error codes, error messages written for a stranger, and a diagnostics endpoint that returns redacted config, dependency check results, versions and circuit-breaker states as a single JSON blob that support can ask them to paste. Second, I build a support bundle script — one command that produces a single archive containing redacted logs for the last N hours, a metrics snapshot from a set of fixed queries, Kubernetes events, and a manifest listing every file and why it is there. Redaction happens at generation, not transmission, so the bundle itself contains no secrets or PII and their security team can inspect it before it leaves. Third, I ship dashboards as code in the Helm chart with provisioned ConfigMaps, so even though I cannot see them, I know exactly what they show and I can talk someone through them panel by panel. Fourth, I deploy a blackbox prober inside their network from week one, because the most common dispute is "your service cannot reach X" and a continuous probe with baseline history settles it without argument. Fifth, I route alerts to their pager and their ticketing system from day one rather than at handover, and I ask for a weekly export or a read-only Grafana viewer account — often refused for the SIEM but granted for a metrics dashboard, and metrics are usually enough.

How do you prevent alert fatigue?

By treating every page as an interruption of a human life with a real cost, and designing accordingly. The biggest single lever is alerting on symptoms rather than causes: page on the user-visible condition — error budget burning fourteen times too fast, p99 latency past the SLO threshold, the oldest queue message older than the freshness target — not on CPU above eighty per cent or a pod restarting, because those are frequently the system working correctly. There are three legitimate exceptions where a cause alert is right: conditions certain to cause an outage that a symptom alert would catch too late, such as a disk filling or a certificate expiring in fourteen days; conditions masked by redundancy, such as a replica pair down to one or a backup not run in forty-eight hours; and security events. Beyond that: use multi-window burn-rate alerts so blips do not page and sustained problems do; set `for` durations of roughly twice the normal recovery time to kill flapping; use inhibition rules so one root cause produces one page rather than forty; and give every alert a severity

> with a real definition and a runbook link that lists the first three things to check, written in the same pull request as the alert. Then run a weekly review of every alert that fired and ask one question of each: was it actionable? If not, delete it or demote it to a ticket. That meeting is the single most effective fatigue cure I know of. Finally, I always deploy a dead man's switch — a watchdog alert that fires permanently and whose absence pages an external heartbeat service — because a monitoring system that dies silently takes the entire alerting capability with it and nobody notices until a customer phones.

> **What makes a postmortem useful, and what makes action items actually get done?**
>
> A postmortem is useful when it changes the system, and it changes the system only if people tell the truth, which they only do if the outcome is not blame. That is not sentimentality — it is epistemology: in a blaming culture people withhold exactly the information you needed. Concretely, a useful postmortem has a timeline built from the incident channel rather than reconstructed from memory, because memory is unreliable and the differences matter; it states impact in business terms with real numbers; it identifies contributing factors plural rather than a single root cause, because distributed system failures always have several; it names what went well and, separately, what was lucky, because luck is not a control and should be made explicit; and it bans counterfactuals — "if only she had checked the dashboard" is hindsight, whereas "the dashboard did not surface queue age" is analysis. For action items, three rules in my experience determine whether they happen. They must be owned by a named individual, never a team, because a team owns nothing. They must have a date and live in the same backlog with the same visibility as feature work. And there must be no more than about seven of them with at most two P0s — a postmortem with twenty-three action items reliably produces zero completed items, while one with four produces four. Then review completion at the start of the next postmortem; "we agreed to do this after the last incident, we did not, and here we are again" is the most persuasive prioritisation argument ever invented.

Instrumentation Checklist and Lab Plan

Two artefacts to take away: a checklist you can run against any service before it goes live in a customer environment, and a seven-part lab that builds the whole stack from nothing on a laptop. Do the lab once properly and the checklist becomes muscle memory.

Pre-deployment instrumentation checklist

LOGGING

- [ ] All logs are JSON to stdout. No files written by the application, no log shipping from inside the process.

- [ ] Every line carries: timestamp (ISO 8601 UTC), level, service, version, environment, trace_id, span_id, request_id, tenant_id.
- [ ] A redaction processor strips secrets by key name and scrubs emails, card numbers and JWTs by pattern, as defence in depth.
- [ ] No request bodies, no response bodies, no file names, no prompts, no PII — audited field by field, not assumed.
- [ ] Route templates are logged, never raw paths with identifiers.
- [ ] Every failure mode has a stable error code, and a one-page code-to-action table exists.
- [ ] Retried-then-succeeded is WARN plus a success INFO with `retries=N`, never ERROR.
- [ ] Log level is configurable at runtime without a restart, and auto-reverts after a time box.
- [ ] Sampling keeps 100% of errors and slow requests, records `sample_rate` in the event, and guarantees per-tenant coverage.

METRICS

- [ ] RED metrics on every endpoint and, separately, on every outbound dependency.
- [ ] USE metrics on every pool: connections, threads, queues, memory, disk.
- [ ] Histogram buckets chosen for this workload's actual latency range, straddling the SLO.
- [ ] Cardinality calculated and documented: labels × values × buckets. No unbounded labels.
- [ ] `build_info` exposed with version and revision.
- [ ] Business metrics exist, not just technical ones: documents processed, invoices matched, tokens spent.
- [ ] Certificate expiry, licence expiry and quota headroom exposed as gauges.
- [ ] Circuit breaker state exposed as a gauge.
- [ ] `metric_relabel_configs` and `sample_limit` set defensively on the scrape side.

TRACING

- [ ] Auto-instrumentation enabled for HTTP, database, messaging and gRPC.

- [ ] Manual spans for business operations with business attributes.
- [ ] Context propagation verified across every queue, thread pool and scheduled job — tested, not assumed.
- [ ] Multiple propagators configured: tracecontext, baggage, b3multi, jaeger.
- [ ] No PII, no secrets, no raw SQL literals in span attributes; DB statement sanitiser enabled.
- [ ] Tail sampling in the Collector: all errors, all slow traces, a named debug tenant, a small percentage of the rest.
- [ ] Health-check endpoints excluded from tracing.

ALERTING AND OPERATIONS

- [ ] SLIs defined with an explicit denominator, agreed in writing with the customer.
- [ ] Multi-window multi-burn-rate alerts implemented for availability and latency.
- [ ] Every alert has a severity, an owner, a runbook URL, a dashboard URL and a business-impact description.
- [ ] Every runbook lists the first three diagnostic commands and a safe remediation.
- [ ] Alerts route to the customer's pager and ticketing system, tested end to end with a real page.
- [ ] Dead man's switch deployed and wired to an external heartbeat.
- [ ] Inhibition rules prevent alert storms from a single cause.
- [ ] One dashboard per service, provisioned as code, fitting one screen, with deploy annotations.

CUSTOMER-ENVIRONMENT SPECIFICS

- [ ] Backend destination confirmed, ingest volume estimated, cost ownership agreed in writing.
- [ ] Egress path confirmed: direct, proxy, or none. Collector configured accordingly with a local fallback.
- [ ] Private CA trusted; TLS verification never disabled.
- [ ] Data classification document written and approved; sub-processors listed in the DPA.
- [ ] Startup self-test implemented with per-check remediation text; fails fast on required dependencies.

- [ ] Diagnostics endpoint implemented and documented.
- [ ] Support bundle script shipped and tested by the customer's security team.
- [ ] Blackbox probes deployed inside the customer network for every external dependency.
- [ ] Handover acceptance test scheduled with L1, two weeks before departure.

Lab plan — build the whole stack

1. **Lab 1 — Structured logging and correlation (2 hours)**
   Write a small FastAPI service with two endpoints, one of which calls the other over HTTP. Configure structlog with the processor chain from this guide. Add correlation middleware that accepts or generates a request ID, propagates it over the internal HTTP call, and echoes it in the response header. Verify with `curl -v` that the same ID appears in both services' logs and in the response. Then deliberately log a dictionary containing a password and an email address and confirm your redaction processor catches both. Success criterion: `docker compose logs | jq 'select(.request_id=="X")'` returns the full story of one request across both services.

2. **Lab 2 — Prometheus, PromQL and cardinality (3 hours)**
   Add `prometheus_client` with a request counter, a latency histogram with buckets you chose deliberately, and an in-flight gauge. Run Prometheus in Docker scraping it. Write and verify these queries by hand: request rate by route; error ratio; p50/p95/p99 from the histogram; the same queries aggregated across two replicas. Then deliberately add a `user_id` label, drive 10,000 distinct values through it with a load generator, and watch `prometheus_tsdb_head_series` climb. Fix it with `metric_relabel_configs` and confirm the series count drops. Success criterion: you can explain, from your own graph, why `rate(sum(x))` is wrong and `sum(rate(x))` is right.

3. **Lab 3 — Grafana dashboards as code (2 hours)**
   Build the five-row service dashboard described in this guide. Add template variables for environment and service. Use `$__rate_interval` everywhere. Add a deploy annotation driven by `changes(build_info[ ... ])` and trigger it by restarting your service with a new `APP_VERSION`. Export the JSON, put it in a ConfigMap or provisioning file, delete the dashboard from the UI, restart Grafana, and confirm it comes back. Success criterion: the dashboard exists only in git and still appears after a clean Grafana restart.

4. **Lab 4 — OpenTelemetry end to end (4 hours)**
   Instrument both services with auto-instrumentation plus manual spans. Stand up an OpenTelemetry Collector with the full pipeline from this guide: memory_limiter, resource, attributes scrubbing, health-check filtering, spanmetrics, tail_sampling, batch. Export traces to Tempo or Jaeger and metrics to Prometheus. Add a RabbitMQ or Redis queue between the

services and make the trace span the queue boundary by injecting and extracting headers — then deliberately remove the injection and observe the trace break, so you recognise the symptom. Configure tail sampling to keep all errors and 10% of the rest, and verify by generating 100 requests of which 5 fail. Success criterion: one trace shows HTTP → queue → consumer → database as a single connected tree, and your sampling keeps exactly the traces you intended.

5

Lab 5 — SLOs and burn-rate alerting (3 hours)

Define an availability SLI and a latency SLI for your service. Write the recording rules for 5m, 30m, 1h, 6h and 1d windows. Write the four burn-rate alerts. Stand up Alertmanager with routing by severity, inhibition rules and a webhook receiver you can watch. Then inject a controlled failure — a middleware that returns 500 for a configurable percentage of requests — and tune the failure rate until each alert fires in turn. Measure how long each takes to fire and to resolve. Success criterion: a 2% error rate does not page, a 20% error rate pages within three minutes, and both resolve within five minutes of the injection stopping.

6

Lab 6 — Debugging under pressure (3 hours)

Introduce four faults, one at a time, and diagnose each using only your dashboards, logs and traces — no reading your own source code. (a) An N+1 query: load 200 rows with lazy loading and find it in a flame graph. (b) A connection pool exhaustion: set pool size to 5, add a 3-second artificial delay in a dependency, and watch the cascade; then add a pool acquisition timeout and a latency-aware circuit breaker and prove the difference. (c) A memory leak: append to a module-level list per request, find it with `tracemalloc` or a heap snapshot. (d) A CPU hot spot: add a catastrophically backtracking regex, find it with `py-spy record`. Write a two-paragraph postmortem for each using the template. Success criterion: median time to diagnosis under fifteen minutes by the fourth fault.

7

Lab 7 — The customer-environment simulation (4 hours)

Reconfigure everything as if you were deployed on site. Put a forward proxy in front of all egress and configure the Collector to use it. Generate a private CA, sign your internal TLS certificates with it, and make every component trust it without disabling verification anywhere. Implement the startup self-test with ten checks and verify it fails fast and loudly when you break DNS, when you break the firewall rule, and when you present an untrusted certificate — and that each failure prints useful remediation text. Write and run the support bundle script, then read the output as if you were the customer's security reviewer and remove anything you would not want to send. Finally, write the runbook for one alert and hand the whole thing to a colleague with no context: can they, from your artefacts alone, diagnose a fault you have injected? Success criterion: they can, without asking you a question.

✓ **The one habit that compounds**

For every incident you are involved in, ask afterwards: "what telemetry would have made this diagnosis take five minutes instead of two hours?" Then add it. Not as a backlog item — in the same week, while the memory of the pain is fresh. Over a couple of years this single habit builds an instrumentation instinct that is far more valuable than knowing any particular tool, and it is directly visible to customers as the difference between an engineer who guesses and one who knows.